UNIVERSITAS GUNADARMA

FAKULTAS TEKNOLOGI INDUSTRI

PENGEMBANGAN SISTEM DETEKSI BAHASA ISYARAT SIBI BERBASIS WEB MENGGUNAKAN YOLOV8 DENGAN METODOLOGI CRISP-DM

SKRIPSI:

Nama : Evandri Ridho Hasmono

NPM : 50422473

Program Studi : Informatika

Pembimbing : Dr. Rina Noviana, S.Kom., MMSI.

Diajukan Guna Melengkapi Sebagian Syarat

Dalam Mencapai Gelar Sarjana Strata Satu (S1)

JAKARTA

2026

PERNYATAAN ORISINALITAS DAN PUBLIKASI

Saya yang bertanda tangan di bawah ini:

Nama	:	Evandri Ridho Hasmono
NPM	:	50422473
Judul Penulisan Ilmiah	:	Pengembangan Sistem Deteksi Bahasa Isyarat SIBI Berbasis Web Menggunakan YOLOv8 Dengan Metodologi CRISP-DM
Tanggal Sidang	:	04 Agustus 2026
Tanggal Lulus	:	04 Agustus 2026

Menyatakan bahwa tulisan ini di atas merupakan hasil karya saya sendiri dan dapat dipublikasikan sepenuhnya oleh Universitas Gunadarma. Segala kutipan dalam bentuk apa pun telah mengikuti kaidah dan etika yang berlaku. Semua hak cipta dari logo serta produk yang disebut dalam buku ini adalah masing-masing pemegang haknya, kecuali disebutkan lain. Mengenai isi dan tulisan merupakan tanggung jawab Penulis, bukan Universitas Gunadarma.

Demikianlah pernyataan ini dibuat dengan sebenarnya dan dengan penuh kesadaran.



(Evandri Ridho Hasmono)

LEMBAR PENGESAHAN

Judul Skripsi	:	Pengembangan Sistem Deteksi Bahasa Isyarat SIBI Berbasis Web Menggunakan YOLOv8 Dengan Metodologi CRISP-DM
Nama Mahasiswa	:	Evandri Ridho Hasmono
NPM	:	50422473
Tanggal Lulus	:	04 Agustus 2026

PANITIA UJIAN

NO	NAMA	KEDUDUKAN
1.	Prof. Dr. Ravi Ahmad Salim	Ketua
2.	Prof. Dr. Wahyudi Priyono	Sekretaris
3.	Dr. Rina Noviana, SKom., MMSI.	Anggota
4.	Dr. Astie Darmayantie, ST., MMSI., MSc.	Anggota
5.	Dr. Ana Kurniawati, ST., MMSI.	Anggota

Menyetujui,

Pembimbing **Bagian Sidang Ujian**

(Dr. Rina Noviana, S.Kom., MMSI) **(Dr. Edi Sukirman, S.Si., MM., M.I.Kom)**

ABSTRAK

Evandri Ridho Hasmono. 50422473

PENGEMBANGAN SISTEM DETEKSI BAHASA ISYARAT SIBI BERBASIS WEB MENGGUNAKAN YOLOV8 DENGAN METODOLOGI CRISP-DM

Skripsi. Program Studi Informatika, Fakultas Teknologi Industri, Universitas Gunadarma, 2026.

Kata kunci: SIBI, YOLOv8, deteksi objek, CRISP-DM, Bahasa Isyarat

(xii + 72 + Lampiran)

Bahasa isyarat merupakan sarana komunikasi utama bagi penyandang tunarungu di Indonesia, namun keterbatasan pemahaman masyarakat umum terhadap Sistem Isyarat Bahasa Indonesia (SIBI) masih menjadi hambatan komunikasi yang signifikan, terutama karena minimnya fasilitas penerjemahan bahasa isyarat secara langsung di ruang publik maupun lingkungan pendidikan. Penelitian ini bertujuan untuk membangun sistem deteksi abjad SIBI berbasis web yang mampu mengenali isyarat tangan secara langsung menggunakan algoritma You Only Look Once versi 8 (YOLOv8), yang dipilih karena keunggulannya dalam kecepatan pemrosesan dan akurasi deteksi objek secara langsung. Metodologi penelitian yang digunakan adalah Cross Industry Standard Process for Data Mining (CRISP-DM), yang terdiri atas tahap pemahaman bisnis, pemahaman data, persiapan data, pemodelan, evaluasi, dan penerapan. Model deteksi objek dilatih menggunakan kumpulan data citra abjad SIBI yang telah melalui proses anotasi dan augmentasi, kemudian diintegrasikan ke dalam aplikasi berbasis web agar dapat diakses secara luas oleh pengguna tanpa memerlukan instalasi perangkat lunak khusus. Hasil penelitian menunjukkan bahwa sistem berhasil diterapkan dan mampu mendeteksi abjad SIBI secara langsung melalui antarmuka web dengan baik, dengan tingkat akurasi yang memadai untuk mendukung skenario pembelajaran maupun komunikasi sehari-hari. Sistem ini diharapkan dapat menjadi alat bantu pembelajaran dan komunikasi bagi masyarakat umum maupun penyandang tunarungu, sekaligus berkontribusi dalam meningkatkan aksesibilitas dan inklusivitas komunikasi di Indonesia.

Daftar Pustaka (2020-2025).

ABSTRACT

Evandri Ridho Hasmono. 50422473

DEVELOPMENT OF A WEB-BASED SIBI SIGN LANGUAGE DETECTION SYSTEM USING YOLOV8 WITH THE CRISP-DM METHODOLOGY

Undergraduate Thesis. Informatics Study Program, Faculty of Industrial Technology, Gunadarma University, 2026.

Keyword: SIBI, YOLOv8, object detection, CRISP-DM, Sign Languange

(xii + 72 + Attachment)

Sign language is the primary means of communication for deaf and hard-of-hearing individuals in Indonesia, yet the general public's limited understanding of the Indonesian Sign Language System (Sistem Isyarat Bahasa Indonesia/SIBI) remains a significant communication barrier, particularly due to the scarcity of instant sign language translation facilities in public spaces and educational environments. This research aims to develop a web-based SIBI alphabet detection system capable of recognizing hand signs instantly using the You Only Look Once version 8 (YOLOv8) algorithm, which was selected for its superior processing speed and object detection accuracy in instant applications. The research methodology employed is the Cross Industry Standard Process for Data Mining (CRISP-DM), consisting of business understanding, data understanding, data preparation, modeling, evaluation, and deployment stages. The object detection model was trained using a dataset of SIBI alphabet images that had undergone annotation and augmentation processes, and was then integrated into a web-based application to ensure broad accessibility without requiring specialized software installation. The results show that the system was successfully deployed and able to detect SIBI alphabets instantly through the web interface effectively, achieving an accuracy level adequate to support both learning and everyday communication scenarios. This system is expected to serve as a learning and communication tool for both the general public and deaf and hard-of-hearing individuals, while contributing to improved communication accessibility and inclusivity in Indonesia.

References (2020-2025).

KATA PENGANTAR

Segala puji dan syukur penulis panjatkan ke hadirat Allah Subhanahu Wata'ala atas limpahan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan penyusunan Tulisan Skripsi yang berjudul "Pengembangan Sistem Deteksi Bahasa Isyarat SIBI Berbasis Web Menggunakan YOLOv8 Dengan Metodologi CRISP-DM" dengan baik dan lancar. Tulisan Skripsi ini disusun sebagai salah satu syarat untuk menyelesaikan pendidikan pada jenjang sarjana muda di Program Studi Informatika, Fakultas Teknologi Industri, Universitas Gunadarma. Penulis menyadari sepenuhnya bahwa tanpa dukungan, bimbingan, dan bantuan dari berbagai pihak, penyusunan tulisan ini tidak akan dapat terselesaikan dengan baik. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan rasa hormat dan terima kasih yang sebesar-besarnya kepada semua pihak yang telah membantu dan membimbing penulis dalam menyusun Tulisan Skripsi ini, di antaranya kepada:

1. Prof. Dr. E. S. Margianti SE., MM., selaku Rektor Universitas Gunadarma.
2. Prof. Dr.-Ing. Adang Suhendra, S.Si., SKom, MSc., Dekan Fakultas Teknologi Industri Universitas Gunadarma.
3. Prof. Dr. Lintang Yuniar Banowosari, S.Kom., M.Sc., selaku Ketua Program Studi Informatika Fakultas Teknologi Industri Universitas Gunadarma.
4. Dr. Edi Sukirman, S.Si., MM., M.I.Kom., selaku Kepala Bagian Sidang Ujian Universitas Gunadarma.
5. Dr. Rina Noviana, S.Kom., MMSI., selaku Dosen Pembimbing pada Skripsi ini.
6. Yanti Tri Astuti dan Kulup Sugijarto tercinta yang telah memberikan doa dan support terbaik dalam penulisan skripsi ini.
7. Sandy, Arya, dan Nickson yang telah membantu sekaligus menyemangati penulisan skripsi.

Karena keterbatasan ilmu dan pengalaman yang dimiliki, saya mengharapkan kritik dan saran yang membangun dari pembaca tulisan ini. Saya juga berharap penulian ini dapat bermanfaat dan menambah wawasan pembaca dan lebih-lebih pembaca mampu menghasilkan inovasi-inovasi yang baru dari penulisan ini.

Jakarta, Juli 2026,



(Evandri Ridho Hasmono)

DAFTAR ISI

UNIVERSITAS GUNADARMA ..... **i**
PERNYATAAN ORISINALITAS DAN PUBLIKASI ..... **ii**
LEMBAR PENGESAHAN ..... **iii**
ABSTRAK ..... **iv**
ABSTRACT ..... **v**
KATA PENGANTAR ..... **vi**
DAFTAR ISI ..... **viii**
DAFTAR TABEL ..... **x**
DAFTAR GAMBAR ..... **xi**
DAFTAR LAMPIRAN ..... **xii**
1. PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Ruang Lingkup 3
1.4 Tujuan Penelitian 4
1.5 Sistematika Penulisan 4
2. TINJAUAN PUSTAKA **5**
2.1 *Artificial Intelligence* (AI) 5
2.2 *Machine Learning* (ML) 6
2.2.1 *Supervised Learning* 8
2.2.2 *Unsupervised Learning* 8
2.2.3 *Reinforcement Learning* 9
2.3 *Deep Learning* (DL) 9
2.4 *Computer Vision* (CV) 10
2.5 YOLO 12
2.5.1 YOLOv8 13
2.6 SIBI 15
2.7 CRISP-DM 16
2.8 *Roboflow* 18
2.9 *Kaggle* 19
2.10 Evaluasi Model 21

2.10.1 *Confusion Matrix* 21
2.10.2 Precision dan Recall 21
2.10.3 Mean Average Precision (mAP) 22
2.11 Python 22
2.12 Docker 23
2.13 *Virtual Privater Server* 25
2.14 Penelitian Terdahulu 26
3. METODE PENELITIAN 29
3.1 Tahapan Penelitian 29
3.1.1 *Business Understanding* 32
3.1.2 *Data Understanding* 34
3.1.3 *Data Preparation* 35
3.1.4 *Modeling* 37
3.1.5 *Evaluation* 39
3.1.6 *Deployment* 40
4. HASIL DAN PEMBAHASAN 42
4.1 Hasil *Business Understanding* 42
4.2 Hasil *Data Understanding* 42
4.3 Hasil *Data Preparation* 44
4.3.1 Hasil *Anotate* 45
4.3.2 Hasil *Data Splitting* 47
4.3.3 Hasil *Preprocessing* 48
4.3.4 Hasil *Data Augmentasi* 48
4.4 Hasil *Modeling* 52
4.5 Hasil *Evaluation* 56
4.6 Hasil *Deployment* 60
4.6.1 *Deployment* 60
4.6.2 Pengujian 65
5. PENUTUP 68
5.1 Kesimpulan 68
5.2 Saran 68
DAFTAR PUSTAKA 69
LAMPIRAN LISTING PROGRAM 1
LAMPIRAN OUTPUT PROGRAM 9

DAFTAR TABEL

Tabel 2. 1 Tabel Penelitian Terdahulu 26
Tabel 3. 1 Sampel Abjad SIBI 29
Tabel 3. 2 Penjelasan Metrik Evaluasi 39
Tabel 4. 1 Sampel Hasil Anotate 45
Tabel 4. 2 Sample Hasil Augmentasi 50
Tabel 4. 3 Perbandingan Epoch 57
Tabel 4. 4 Tabel pengujian 66

DAFTAR GAMBAR

Gambar 2. 1 Artificial Intelligence (Investopedia, 2026) 6
Gambar 2. 2 Klasifikasi Machine Learning 7
Gambar 2. 3 Deep Learning 10
Gambar 2. 4 Contoh deteksi Bahasa Isyarat 12
Gambar 2. 5 Halaman Beranda Ultralytics 13
Gambar 2. 6 Abjad SIBI (Yayasan Peduli Kasih ABK, 2023) 16
Gambar 2. 7 CRISP-DM (Nurfatih, Nurmalasari dan Prakarsyah, 2024) 18
Gambar 2. 8 Tampilan Beranda Roboflow 19
Gambar 2. 9 Logo Kaggle 20
Gambar 2. 10 Confusion Matrix 21
Gambar 2. 11 Tampilan Beranda Python 23
Gambar 2. 12 Logo Docker 25
Gambar 2. 13 Dashboard IDCloud 26
Gambar 3. 1 Diagram CRISP-DM 32
Gambar 3. 2 Aristektur YOLOv8 Penelitian 38
Gambar 4. 1 Dataset Abjad 43
Gambar 4. 2 Hasil Dataset Teranotasi 44
Gambar 4. 3 Hasil Data Splitting 47
Gambar 4. 4 Pengaturan Preprocessing 48
Gambar 4. 5 Pengaturan Augmentasi 49
Gambar 4. 6 Jumlah Dataset setelah proses Augmentasi 51
Gambar 4. 7 Tampilan Halaman Antigravity 53
Gambar 4. 8 Struktur Direktori Training 53
Gambar 4. 9 Skrip Pelatihan model 54
Gambar 4. 10 Proses Training Model 55
Gambar 4. 11 Hasil Pelatihan Model 56
Gambar 4. 12 Grafik Hasil Pelatihan 57
Gambar 4. 13 Confusion Matrix Abjad SIBI 58
Gambar 4. 14 Hasil model abjad SIBI 59
Gambar 4. 15 Arsitektur sistem 60
Gambar 4. 16 Struktur Folder 61
Gambar 4. 17 Implementasi Load Model 62
Gambar 4. 18 Implementasi Run Inference 62
Gambar 4. 19 Code docker image 63
Gambar 4. 20 proses git pull 64
Gambar 4. 21 Proses Build 64
Gambar 4. 22 Tampilan website 65

DAFTAR LAMPIRAN

LAMPIRAN LISTING PROGRAM……L-1
LAMPIRAN OUTPUT PROGRAM……L-9

1. PENDAHULUAN

1.1 Latar Belakang

Komunikasi merupakan kebutuhan mendasar setiap manusia, namun bagi penyandang tunarungu hal ini menjadi tantangan tersendiri. Berdasarkan laporan World Health Organization, gangguan pendengaran saat ini memengaruhi sekitar 1,59 miliar orang di seluruh dunia dan diperkirakan akan meningkat hingga 2,5 miliar pada tahun 2050 (Jia & Li, 2024). Di Indonesia, Sistem Isyarat Bahasa Indonesia (SIBI) telah ditetapkan sebagai standar komunikasi resmi bagi penyandang tunarungu, namun penggunaannya masih sangat terbatas di masyarakat umum sehingga kesenjangan komunikasi tetap terjadi (Suharjito et al., 2021).

Upaya penelitian untuk mengotomasi pengenalan SIBI telah dilakukan sebelumnya. (Suharjito et al., 2021) mengembangkan sistem pengenalan SIBI menggunakan Convolutional Neural Network (CNN) berbasis model Inflated 3D yang dikombinasikan dengan transfer learning, dan berhasil mencapai akurasi pengujian sebesar 97,50%. Meskipun akurasinya tinggi, penelitian tersebut belum mengimplementasikan sistem dalam platform yang dapat diakses secara real-time oleh pengguna umum. (Jia & Li, 2024) kemudian membuktikan bahwa YOLOv8 unggul dibandingkan model YOLO sebelumnya dalam hal kecepatan dan akurasi deteksi bahasa isyarat, menjadikannya kandidat kuat untuk sistem yang responsif.

Berdasarkan kesenjangan tersebut, penelitian ini mengimplementasikan YOLOv8 untuk deteksi isyarat SIBI yang diintegrasikan ke dalam aplikasi berbasis web menggunakan ReactJs sebagai antarmuka pengguna dan FastAPI sebagai backend. Dataset yang digunakan bersumber dari Kaggle yang dikembangkan oleh M. Lanang Afkaar berjudul Datasets Lemlitbang SIBI Alphabets, dengan proses anotasi, preprocessing, dan augmentasi data dilakukan menggunakan platform Roboflow.

Pemilihan YOLOv8 dalam penelitian ini didasarkan hasil komparasi yang dilakukan oleh (Jia & Li, 2024), yang menguji beberapa varian YOLO yaitu YOLOv5, YOLOv6, YOLOv7, dan YOLOv8 pada *dataset* bahasa isyarat yang sama. Hasil pengujian menunjukkan bahwa akurasi YOLOv5, YOLOv6, dan

YOLOv7 hanya berkisar 0,61 hingga 0,96 kali dari akurasi YOLOv8, sementara jumlah parameter YOLOv8 hanya sedikit lebih besar dibandingkan model-model tersebut. YOLOv8 juga telah mengadopsi arsitektur *decoupled head* yang memisahkan proses klasifikasi dan deteksi, serta pendekatan *anchor-free* yang menghilangkan kebutuhan *anchor box* sehingga mengurangi waktu komputasi.

Proses *preprocessing* data merupakan tahapan krusial dalam mempersiapkan dataset sebelum digunakan untuk pelatihan model. Pada penelitian ini, proses tersebut dilakukan melalui platform Roboflow untuk menyeragakan dimensi dan memperbaiki orientasi gambar secara otomatis, guna menjaga kestabilan dan keefektifan proses pelatihan (Fatahna et al., 2025). Selain itu, dataset juga diperkaya melalui serangkaian teknik augmentasi. Langkah ini bertujuan untuk memperbanyak variasi data sekaligus meningkatkan kemampuan generalisasi model, sehingga sistem dapat lebih tangguh dalam mengenali isyarat tangan pada berbagai kondisi visual yang beragam (Fatahna et al., 2025).

Penelitian ini menggunakan metodologi CRISP-DM (Cross-Industry Standard Process for Data Mining) sebagai kerangka utama, yang mencakup enam tahap, yaitu Business Understanding, Data Understanding, Data preparation, Modeling, Evaluation, dan Deployment (Schröer et al., 2021). Pendekatan serupa yang mengombinasikan metodologi CRISP-DM dengan model YOLO serta integrasi ke dalam applikasi berbasis web telah diterapkan oleh (Nabila et al., 2025) Pada penelitian deteksi dan klasifikasi sampah menggunakan YOLOv11. Tahap Deployment pada penelitian ini diwujudkan dalam bentuk integrasi model YOLOv8 ke dalam applikasi berbasis web menggunakan ReactJs dan FastAPI, sehingga hasil deteksi dapat diakses oleh pengguna.

1.2 Rumusan Masalah

Berdasarkan uraian latar belakang yang telah dijelaskan sebelumnya, maka dapat dirumuskan beberapa permasalahan dalam studi ini sebagai berikut:

1. Bagaimana tingkat akurasi dan peforma model YOLOv8 dalam mendeteksi huruf-huruf SIBI berdasarkan citra gestur tangan?

2. Bagaimana penerapan algoritma YOLOv8 dalam melakukan deteksi huruf SIBI dari gestur tangan?
3. Bagaimana tahapan metodologi CRISP-DM diterapkan dalam proses pengembangan model deteksi huruf SIBI?

Rumusan masalah menjadi dasar dalam perancangan dan pengembangan sistem, sehingga setiap tahapan penelitian diarahkan untuk menjawab tiga pertanyaan tersebut secara terukur dan dapat dipertanggungjawabkan secara ilmiah.

1.3 Ruang Lingkup

Studi ini memiliki beberapa batasan masalah agak tidak menyimpang dan melebar dari rumusan permasalahan diatas. Batasan masalah sebagai berikut:

1. Pengenalan Isyarat tangan statis SIBI yang merepresentasikan 26 huruf alfabet (A – Z). Isyarat dinamis yang melibatkan gerakan berurutan tidak termasuk dalam cakupan penelitian ini.
2. Dataset yang digunakan adalah Datasets Lemlitbang SIBI Alphabets yang dikembangkan oleh M. Lanang Afkaar dan dipublikasikan melalui platform Kaggle, dengan total sebanyak 1.307 citra. Proses anotasi label, preprocessing gambar, serta augmentasi data dilakukan menggunakan platform Roboflow sebelum dataset digunakan untuk pelatihan model, sehingga jumlah dataset meningkat menjadi 3.137 citra.
3. Proses pelatihan model deteksi dilakukan menggunakan arsitektur YOLOv8 (Ultralytics) berbasis Python, dengan memanfaatkan dataset hasil anotasi dan augmentasi sebagaimana dijelaskan pada point dua.
4. Sistem dibangun sebagai aplikasi berbasis web dengan ReactJs sebagai frontend, FastAPI sebagai Backend, dan PostgreSQL sebagai Database
5. Pengembangan perangkat lunak pada penelitian ini dilakukan menggunakan Antigravity sebagai *Integrated Development Environment* (IDE).
6. Metrik evaluasi yang digunakan mencakup precision, recall, mean Average Precision (mAP), dan confusion matrix.
7. Pengujian dilakukan pada kondisi pencahayaan indoor standar dan berbagai variasi latar belakang untuk mengevaluasi ketahanan model

dalam kondisi penggunaan sehari-hari, dengan menggunakan kamera perangkat pengguna.

1.4 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah dijelaskan sebelumnya, maka tujuan dari penelitian ini adalah sebagai berikut:

1. Mengukur dan mengevaluasi tingkat akurasi serta peforma model YOLOv8 saat diuji coba untuk mendeteksi isyarat.
2. Mengimplementasikan model YOLOv8 untuk mendeteksi abjad SIBI dari citra tangan.
3. Menerapkan tahapan metodologi CRISP-DM dalam proses pengembangan model deteksi huruf SIBI menggunakan algoritma YOLOv8.

1.5 Sistematika Penulisan

Penulisan skripsi ini terdiri dari lima bab yang disusun secara sistematis. Bab 1 merupakan pendahuluan yang memuat latar belakang, rumusan masalah, ruang lingkup, tujuan penelitian, dan sistematika penulisan. Bab 2 adalah tinjauan pustaka yang menguraikan landasan teori meliputi SIBI, deep learning, arsitektur YOLOv8, serta kajian penelitian-penelitian terdahulu yang relevan. Bab 3 menjelaskan metode penelitian berdasarkan tahapan CRISP-DM, meliputi Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, dan Deployment. Bab 4 menyajikan hasil pelatihan dan evaluasi model YOLOv8 berdasarkan metrik *precision*, *recall*, *mean Average Precision* (mAP), dan *confusion matrix*, analisis performa deteksi pada berbagai kondisi penggunaan melalui antarmuka web. Bab 5 berisi Kesimpulan dari keseluruhan penelitian dan saran untuk pengembangan lebih lanjut di masa mendatang.

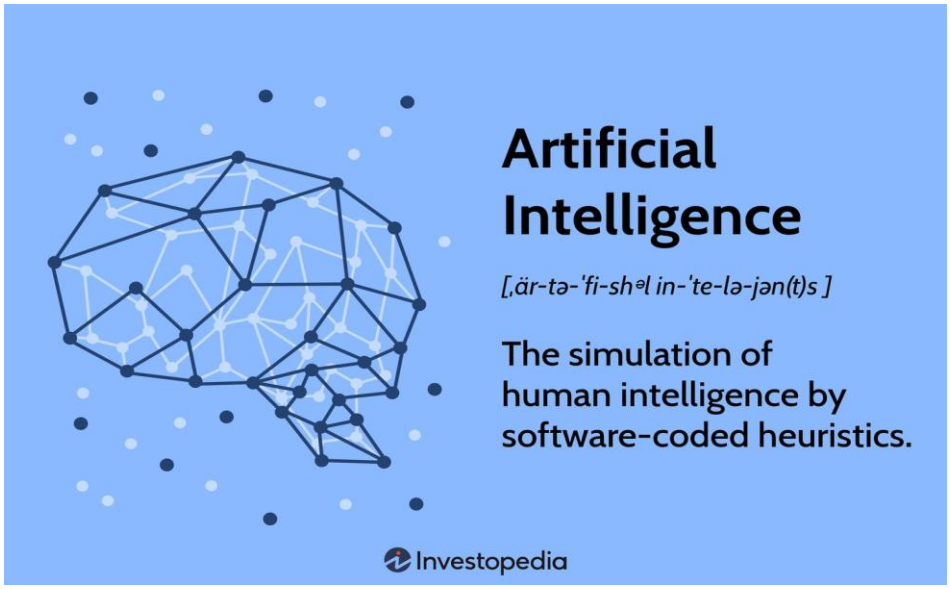
2. TINJAUAN PUSTAKA

2.1 *Artificial Intelligence* (AI)

Artificial Intelligence (AI) atau kecerdasan buatan merupakan cabang ilmu komputer yang berfokus pada pengembangan sistem komputasi cerdas dengan kemampuan yang menyerupai kecerdasan manusia (Sarker, 2022). Secara historis, embrio teknologi ini mulai digagas pada pertengahan abad ke-20 dan secara resmi diperkenalkan melalui Konferensi Dartmouth pada tahun 1956. Seiring dengan peningkatan drastic pada kapasitas komputasi modern serta ketersediaan data yang melimpah, AI telah bertransformasi dari sekadar konsep teoritis menjadi sebuah teknologi fundamental yang mendasari berbagai inovasi mutakhir di industri.

Dalam implementasinya, AI dirancang untuk mengeksekusi beragam tugas kompleks yang secara tradisional memerlukan fungsi kognitif manusia (Collins et al., 2021). Kemampuan kognitif buatan ini mencakup proses pengambilan keputusan berbasis data, pemecahan masalah secara adaptif, pemahaman bahasa alami, hingga pengenalan pola visual melalui visi komputer. Integrasi teknologi AI ke dalam sistem digital terbukti mampu mengotomatisasi proses bisnis, meminimalkan risiko kesalahan manusia, serta meningkatkan efisiensi operasional secara signifikan pada berbagai sektor.

Secara structural, istilah *Artificial Intelligence* merupakan paying besar dari ekosistem kecerdasan buatan yang memayungi beberapa sub-bidang spesifik di dalamnya (Sarker, 2022) Di bawah payung AI, terdapat konsep *Machine Learning* yang berfokus pada kemampuan sistem untuk belajar mandiri dari data tanpa di program secara eksplisit. Lebih spesifik lagi, terdapat sub-materi *Deep Learning* yang memanfaatkan arsitektur jaringan saraf tiruan berlapis untuk menyelesaikan komputasi non-linear yang rumit, termasuk dalam pemrosesan citra digital maupun deteksi objek.



Gambar 2. 1 *Artificial Intelligence* (Investopedia, 2026)

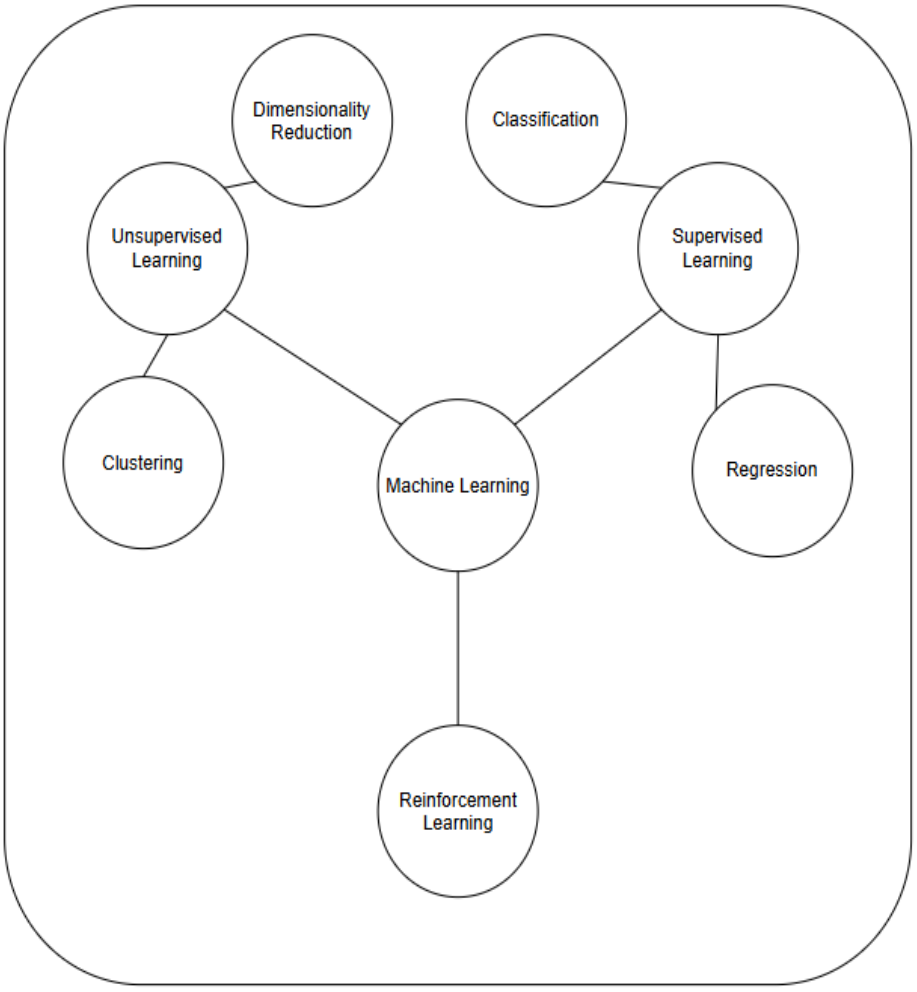
2.2 *Machine Learning* (ML)

Machine Learning (ML) merupakan sub-bidang utama dari kecerdasan buatan yang bertumpu pada algoritma komputasi untuk mengenali pola dari sekumpulan data tanpa memerlukan pemrograman yang eksplisit (Janiesch et al., 2021). Berbeda dengan sistem perangkat lunak konvensional yang beroperasi berdasarkan aturan kaku yang ditulis oleh manusia, ML mengotomatisasi proses pembangunan model analitik dengan menerapkan algoritma yang belajar secara iteratif dari data pelatihan spesifik masalah, sehingga sistem dapat menemukan wawasan tersembunyi dan pola kompleks tanpa diprogram secara eksplisit.

Secara konseptual, algoritma *Machine Learning* umumnya diklasifikasikan ke dalam tiga kategori utama berdasarkan metode pembelajarannya, yaitu *supervised learning*, *unsupervised learning*, dan *reinforcement learning* (Janiesch et al., 2021). Pada *supervised learning*, model komputasi dilatih menggunakan dataset yang sudah memiliki label atau nilai target untuk output, sehingga pasangan data input dan output tersebut digunakan untuk mengkalibrasi parameter model, yang kemudian dapat dipakai memprediksi target pada data baru. Sebaliknya, *unsupervised learning* diterapkan pada data yang tidak berlabel dengan tujuan menemukan struktur yang menarik di dalam data, seperti pengelompokan elemen

dengan properti yang sama (clustering) atau reduksi dimensi (dimensionality reduction). Sementara itu, *reinforcement learning* berfokus pada model yang belajar mencapai suatu tujuan melalui prinsip trial and error untuk memaksimalkan reward, berdasarkan deskripsi keadaan sistem saat ini, tujuan yang ditetapkan, serta daftar tindakan yang diperbolehkan beserta batasan lingkungannya.

Pemanfaatan teknologi *Machine Learning* telah berhasil diterapkan pada berbagai bidang seperti deteksi fraud, credit scoring, analisis next-best offer, pengenalan suara dan gambar, hingga *natural language processing* (Janiesch et al., 2021). Dengan mempelajari komputasi sebelumnya dan mengekstraksi keteraturan dari basis data yang masif, ML membantu menghasilkan keputusan yang andal dan dapat direplikasi.



Gambar 2. 2 Klasifikasi *Machine Learning*

2.2.1 *Supervised Learning*

Supervised learning merupakan metode pembelajaran yang membutuhkan dataset pelatihan berisi pasangan data input beserta label atau nilai target dari output yang diharapkan (Janiesch et al., 2021). Sebagai contoh, dalam kasus prediksi jumlah pengguna aktif suatu platform pada periode tertentu (sebagai variabel target atau *y*), model dilatih menggunakan karakteristik input seperti jumlah produk terjual atau ulasan positif pengguna (disebut sebagai fitur input atau variabel *x*). Pasangan data input dan output pada data pelatihan tersebut kemudian digunakan untuk mengkalibrasi parameter model, sehingga setelah proses pelatihan selesai, model dapat digunakan untuk memprediksi nilai target pada data baru yang belum pernah dilihat sebelumnya.

Berdasarkan jenis keluarannya, *supervised learning* dapat dibedakan lagi menjadi dua jenis permasalahan, yaitu masalah regresi, di mana hasil prediksi berupa nilai numerik (misalnya jumlah pengguna), dan masalah klasifikasi, di mana hasil prediksi berupa kelas kategori tertentu, seperti membedakan pengguna menjadi kategori "hanya melihat-lihat" atau "pembeli" (Janiesch et al., 2021).

2.2.2 *Unsupervised Learning*

Berbeda dengan supervised learning, unsupervised learning diterapkan ketika sistem pembelajaran dituntut untuk mendeteksi pola tanpa adanya label atau spesifikasi yang sudah ditentukan sebelumnya (Janiesch et al., 2021). Dengan demikian, data pelatihan yang digunakan hanya terdiri dari variabel input (x) saja, dengan tujuan menemukan struktur atau informasi menarik yang tersembunyi di dalam data tersebut, seperti pengelompokan elemen-elemen yang memiliki kesamaan properti atau reduksi dimensi data dari ruang berdimensi tinggi menjadi ruang berdimensi lebih rendah.

Salah satu contoh penerapan unsupervised learning yang umum dijumpai adalah penggunaan teknik clustering untuk mengelompokkan pelanggan atau segmen pasar ke dalam beberapa kelompok tertentu, dengan tujuan mendukung strategi komunikasi yang lebih spesifik sesuai karakteristik masing-masing kelompok (Janiesch et al., 2021).

2.2.3 *Reinforcement Learning*

Reinforcement learning memiliki pendekatan yang berbeda dari kedua jenis pembelajaran sebelumnya. Alih-alih menyediakan pasangan data input dan output, sistem pada reinforcement learning mendeskripsikan kondisi sistem saat ini, menentukan tujuan yang ingin dicapai, serta menyediakan daftar tindakan yang diperbolehkan beserta batasan-batasan lingkungan atas setiap kemungkinan hasil dari tindakan tersebut (Janiesch et al., 2021). Model kemudian membiarkan sistem "mengalami" sendiri proses pencapaian tujuan tersebut dengan menerapkan prinsip trial and error untuk memaksimalkan reward yang diperoleh.

Model reinforcement learning telah diterapkan dengan sangat sukses terutama pada lingkungan tertutup seperti permainan, namun juga relevan digunakan pada sistem multi-agen seperti pasar elektronik (Janiesch et al., 2021).

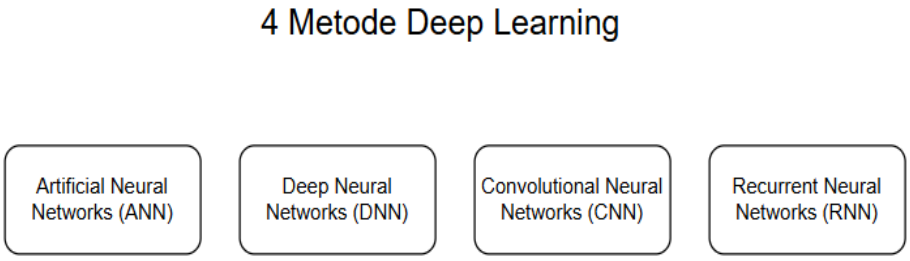
2.3 *Deep Learning* (DL)

Deep Learning merupakan sub-bidang spesifik dan tingkat lanjut dari Machine Learning yang struktur dan fungsinya terinspirasi oleh jaringan saraf biologis pada otak manusia, yang dikenal sebagai jaringan saraf tiruan atau *Artificial Neural Networks* (ANN) (Janiesch et al., 2021). Perbedaan mendasar antara Deep Learning dengan algoritma Machine Learning tradisional terletak pada kedalaman arsitekturnya. Kata "deep" atau "dalam" merujuk pada penggunaan lapisan tersembunyi (*hidden layers*) dalam jumlah yang sangat banyak di antara lapisan input dan output (Alzubaidi et al., 2021). Arsitektur berlapis ini memungkinkan sistem untuk memproses data dengan tingkat abstraksi yang jauh lebih tinggi dan kompleks.

Mekanisme kerja Deep Learning sangat bergantung pada kemampuannya dalam mempelajari representasi fitur secara hierarkis dan otomatis. Pada metode pembelajaran mesin konvensional, ekstraksi fitur seringkali harus dilakukan secara manual oleh manusia (*hand-crafted features*) agar data dapat diproses, sebuah proses yang memakan waktu dan kurang fleksibel. Namun, pada Deep Learning, model komputasi secara mandiri mampu mengekstrak fitur dari data mentah lapis demi lapis tanpa memerlukan banyak campur tangan manusia (Janiesch et al.,

2021). Sebagai contoh, dalam tugas pengenalan objek, lapisan awal akan mengekstrak fitur dasar seperti tepi (*edges*), yang kemudian digabungkan secara bertahap menjadi fitur yang lebih kompleks pada lapisan berikutnya hingga membentuk objek yang menjadi target pengenalan (Alzubaidi et al., 2021).

Keunggulan utama dari Deep Learning adalah performanya yang sangat luar biasa ketika dihadapkan dengan data tidak terstruktur (*unstructured data*) dalam volume yang masif, seperti gambar, teks, video, dan suara. Berkat kapasitas komputasinya yang tinggi, teknologi ini telah menjadi tulang punggung bagi berbagai inovasi mutakhir di bidang kecerdasan buatan, khususnya dalam ranah *Computer Vision* (Alzubaidi et al., 2021). Kemampuannya dalam mengenali pola visual yang rumit secara otomatis menjadikan Deep Learning sebagai standar utama yang digunakan dalam pengembangan algoritma deteksi objek modern (Janiesch et al., 2021).



Gambar 2. 3 Deep Learning

2.4 *Computer Vision* (CV)

Computer Vision (CV) merupakan salah satu bidang penerapan utama dari Deep Learning (DL), selain natural language processing (NLP) serta video/speech recognition (Chai et al., 2021). Pada tahap awal perkembangannya, penelitian di bidang CV masih terkendala oleh keterbatasan memori komputer, CPU, dan GPU, sehingga sebagian besar peneliti pada masa itu lebih banyak mengandalkan pendekatan machine learning konvensional seperti K-means, Naive Bayes, Decision Tree, Random Forest, Haar Classifier, K-Nearest Neighbor (KNN), dan Support Vector Machine (SVM) untuk menyelesaikan permasalahan visual (Chai et al., 2021). Salah satu metode klasik yang berpengaruh dalam sejarah CV adalah

detektor Viola–Jones, yang memanfaatkan fitur wavelet Haar-like dan metode integral graph untuk mendeteksi wajah, dan kemudian disempurnakan oleh Lienhart dan Maydt melalui penambahan fitur Haar rotasi hingga akhirnya diadopsi sebagai Haar classifier dalam pustaka OpenCV (Chai et al., 2021).

Chai et al. (2021)mengelompokkan aplikasi CV berbasis DL ke dalam empat skenario utama, yaitu recognition (yang mencakup image classification dan object detection), visual tracking, semantic segmentation, dan image restoration. Dalam konteks recognition, image classification bertujuan memberikan label pada sebuah input, sedangkan object detection tidak hanya mengklasifikasikan namun juga menentukan lokasi objek target melalui bounding box, yang secara umum diselesaikan melalui pendekatan one-stage (seperti YOLO dan SSD) maupun two-stage (seperti keluarga R-CNN).

Seiring perkembangan literatur, *Convolutional Neural Network* (CNN) kemudian menjadi arsitektur *deep learning* yang paling dominan digunakan dalam CV, terutama setelah keberhasilannya pada kompetisi ImageNet (Chai et al., 2021). Secara umum, arsitektur CNN tersusun atas *convolutional layer*, *pooling layer*, dan *fully connected layer*, di mana *convolutional layer* berfungsi mengekstraksi berbagai *feature map* dari citra menggunakan sejumlah kernel, *pooling layer* berperan mereduksi dimensi *feature map* dan jumlah parameter, sedangkan *fully connected layer* pada umumnya diletakkan di akhir arsitektur dan berfungsi sebagai pengklasifikasi (Chai et al., 2021). Berdasarkan tinjauan terhadap publikasi tahun 2014–2020, teridentifikasi delapan teknik utama yang menjadi model dasar dalam berbagai bidang aplikasi CV, yaitu AlexNet, VGGNet, GoogLeNet & Inception, ResNet, DenseNet, MobileNets, EfficientNet, dan RegNet, yang perkembangannya dikelompokkan ke dalam tiga tahapan, yakni tahap awal (2012–2016), tahap pertengahan (2016–2019), dan tahap terkini (sejak 2019) (Chai et al., 2021).



Gambar 2. 4 Contoh deteksi Bahasa Isyarat

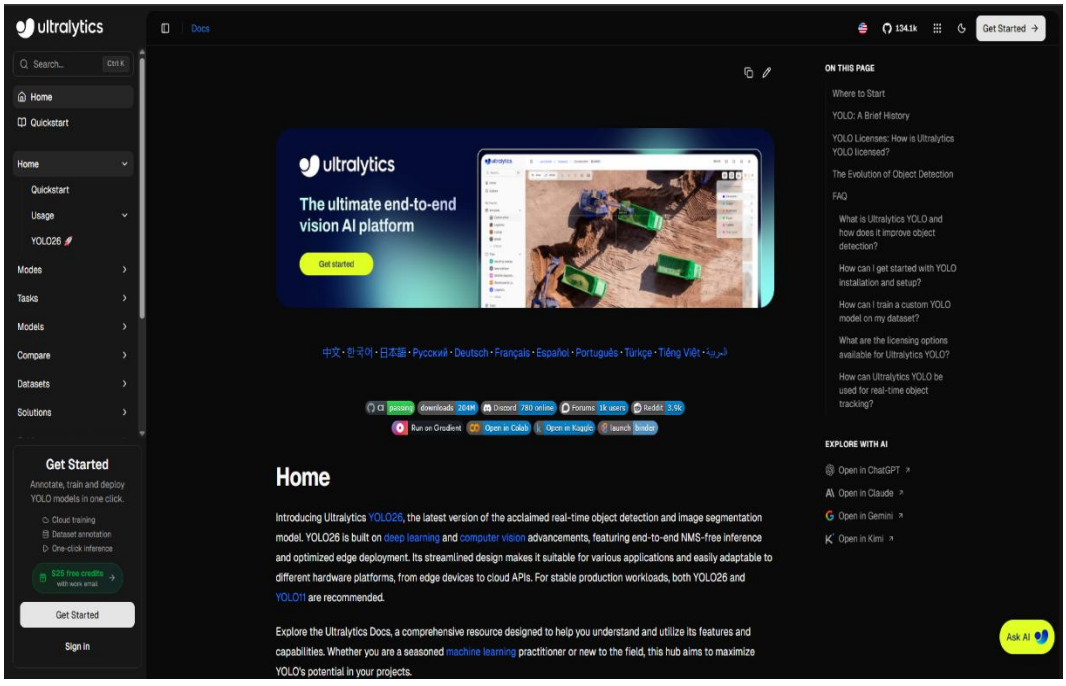
2.5 YOLO

You Only Look Once (YOLO) merupakan salah satu algoritma deteksi objek dengan pendekatan *one-stage detector*, yang pertama kali diperkenalkan dengan memandang permasalahan deteksi objek sebagai persoalan regresi, bukan klasifikasi seperti pada pendekatan *two-stage detector* semacam R-CNN. Keunggulan utama dari pendekatan ini terletak pada kecepatan deteksinya, di mana YOLO versi awal diklaim mampu memproses hingga 45 *frame per second* (fps), bahkan versi cepatnya dapat mencapai 155 fps, meskipun tingkat akurasinya masih berada di bawah sistem deteksi *real-time* lainnya pada masanya (Chai et al., 2021).

Seiring berjalannya waktu, YOLO terus mengalami penyempurnaan melalui beberapa versi lanjutan. YOLOv2 (YOLO9000) memperkenalkan *batch normalization*, resolusi gambar latih yang lebih tinggi, serta *anchor box*, sehingga mampu mendeteksi lebih dari 9000 kategori objek. YOLOv3 kemudian mengganti *backbone* dari Darknet-19 menjadi Darknet-53 yang lebih dalam dengan penambahan operasi *cross-layer* seperti pada ResNet, sehingga akurasinya meningkat meskipun kecepatan pemrosesannya sedikit menurun dibanding pendahulunya. Pengembangan berikutnya, YOLOv4, menambahkan berbagai teknik optimasi seperti *weighted-residual-connection* (WRC), *cross-stage-partial-connections* (CSP), *Mosaic data augmentation*, dan fungsi *loss* CIoU, yang secara

signifikan meningkatkan akurasi tanpa mengorbankan kecepatan deteksi *real-time* (Chai et al., 2021).

Pengembang awal YOLO sempat menghentikan pengembangan lanjutan pada versi ketiga karena mempertimbangkan potensi penyalahgunaan teknologi *computer vision* untuk hal-hal yang meragukan secara etika. Meski demikian, pengembangan seri YOLO tetap berlanjut oleh pihak lain, termasuk oleh perusahaan Ultralytics yang kemudian merilis YOLOv5 dan versi-versi selanjutnya dengan berbagai penyempurnaan arsitektur, termasuk YOLOv8 yang digunakan dalam penelitian ini dan akan dibahas lebih lanjut pada sub-bab berikutnya (Chai et al., 2021).



Gambar 2. 5 Halaman Beranda *Ultralytics*

2.5.1 YOLOv8

YOLOv8 merupakan versi lanjutan dari seri YOLO yang dibuat dan diterbitkan pada Januari 2023 oleh Ultralytics, pengembang yang sebelumnya juga merilis YOLOv5. YOLOv8 dirancang sebagai algoritma *open-source* yang paling banyak digunakan karena mampu menghasilkan hasil pengenalan objek yang

akurat, cepat, dan relatif mudah untuk diimplementasikan dibandingkan versi-versi sebelumnya (Jonathan & Hermanto, 2024).

Secara arsitektur, YOLOv8 terbagi menjadi tiga komponen utama, yaitu *backbone*, *neck*, dan *head*. Bagian *backbone* menggunakan jaringan CSPDarknet53 yang telah dimodifikasi dengan memanfaatkan *cross-stage partial connections* untuk meningkatkan alur informasi antarlapisan sekaligus menaikkan akurasi. Salah satu perubahan penting pada YOLOv8 adalah penggantian blok C3 yang digunakan pada YOLOv5 dengan blok C2f, di mana jumlah *bottleneck* pada modul tersebut turut disesuaikan untuk memperkuat ekstraksi fitur. Bagian *neck* berfungsi menggabungkan *feature map* dari berbagai tahapan *backbone* guna menangkap informasi objek pada berbagai skala ukuran, sedangkan bagian *head* bertanggung jawab melakukan prediksi *bounding box*, *objectness score*, dan probabilitas kelas untuk setiap sel pada *feature map* menggunakan pendekatan *anchor-free*. Secara umum, cara kerja YOLOv8 dapat diringkas menjadi tiga tahap, yaitu pembagian citra masukan menjadi grid, prediksi *bounding box* beserta tingkat kepercayaannya melalui rangkaian lapisan yang saling terhubung, dan penghapusan *bounding box* yang saling tumpang tindih menggunakan algoritma *non-max suppression* (Jonathan & Hermanto, 2024).

Efektivitas YOLOv8 dalam tugas deteksi objek juga telah dibuktikan pada berbagai penelitian penerapan, salah satunya pada studi yang membangun model penghitung jumlah orang melalui deteksi kepala manusia menggunakan dataset sebanyak 2.390 citra. Performa model dievaluasi menggunakan nilai *confusion matrix* beserta metrik evaluasi turunannya, yaitu presisi, akurasi, *recall*, dan *f1-score*, yang menunjukkan bahwa YOLOv8 mampu memberikan hasil deteksi yang cukup akurat untuk kasus penghitungan objek berbasis citra (Dwi Grevika Drantantiyas et al., 2023). Hasil-hasil penelitian semacam ini menjadi dasar pertimbangan digunakannya YOLOv8 dalam penelitian ini, mengingat kemampuannya dalam mendeteksi objek dengan akurasi tinggi serta kecepatan pemrosesan yang memadai untuk diterapkan pada data citra yang digunakan.

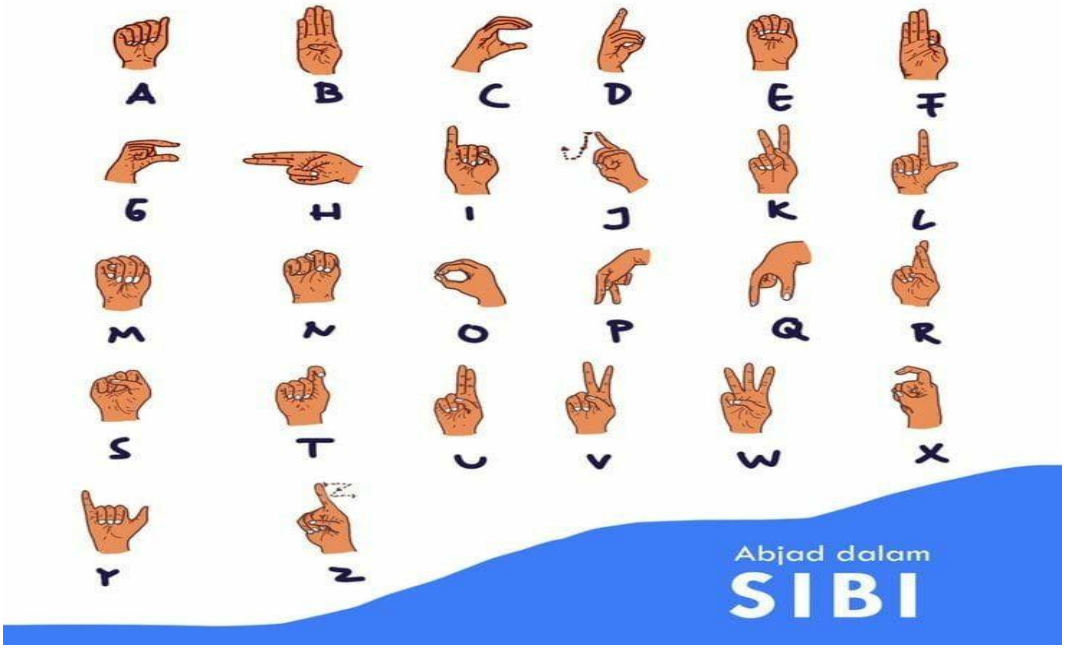
2.6 SIBI

Sistem Isyarat Bahasa Indonesia (SIBI) merupakan bahasa isyarat resmi yang digunakan oleh penyandang tuna rungu di Indonesia untuk berkomunikasi melalui gerakan tangan yang membentuk pola alfabet maupun kata. Pengenalan pola isyarat ini menjadi tantangan tersendiri karena bentuk tangan pada beberapa huruf memiliki kemiripan visual yang tinggi, sehingga sering menyulitkan proses klasifikasi otomatis. Untuk mengatasi hal tersebut, penelitian oleh (Darmatasia, 2021) mengusulkan penambahan filter gradient pada lapisan input citra alfabet SIBI sebelum diproses oleh algoritma Convolutional Neural Network (CNN), dengan tujuan mempertegas bentuk objek tangan agar lebih mudah dikenali oleh model. Hasil penelitian tersebut menunjukkan bahwa penerapan filter gradient mampu meningkatkan akurasi pengenalan dari sekitar 25% (tanpa filter) menjadi sekitar 85%, dan bahkan dapat mencapai 98% seiring bertambahnya jumlah iterasi pelatihan, serta terbukti efektif ketika diuji pada dataset benchmark dengan latar belakang yang kompleks (Darmatasia, 2021).

Pendekatan serupa berbasis CNN juga diterapkan dalam pengembangan sistem penerjemah isyarat tangan SIBI secara real time yang memanfaatkan citra dari webcam. Pada penelitian ini, tahap pre-processing dilakukan dengan mengonversi citra ke ruang warna HSV, dilanjutkan dengan cropping dan thresholding untuk mengisolasi area tangan sebagai region of interest (ROI) agar proses pengenalan lebih akurat (Dwi Nurhayati et al., 2021). Dataset yang digunakan terdiri atas 26 kelas huruf alfabet SIBI ditambah beberapa kelas tambahan seperti spasi dan hapus, dan pengujian model dilakukan pada berbagai kondisi pencahayaan serta jarak pengambilan gambar yang berbeda-beda untuk menguji ketahanan model terhadap variasi lingkungan (Dwi Nurhayati et al., 2021)

Secara historis, SIBI ditetapkan sebagai bahasa isyarat resmi oleh pemerintah melalui Kementerian Pendidikan dan Kebudayaan sejak tahun 1995 dan digunakan secara khusus di lingkungan Sekolah Luar Biasa (SLB) sebagai bahasa pengantar materi pembelajaran bagi siswa tuna rungu (Rachmawati, 2020). Berbeda dengan bahasa isyarat pada umumnya yang tumbuh secara alami dari interaksi sosial komunitas Tuli, SIBI justru dirancang dengan mengacu pada struktur tata bahasa

Indonesia lisan, sehingga penggunaannya cenderung baku dan mengikuti urutan kalimat bahasa Indonesia, sementara sebagian kosakatanya diadaptasi dari bahasa isyarat Amerika (Rachmawati, 2020). Karakteristik inilah yang membedakan SIBI secara mendasar dari Bahasa Isyarat Indonesia (BISINDO), yang tumbuh alami di kalangan Tuli dan dianggap lebih merepresentasikan budaya Tuli lokal, sehingga di lapangan kerap muncul perdebatan mengenai bahasa isyarat mana yang lebih relevan digunakan sebagai alat komunikasi sehari-hari (Rachmawati, 2020). Meskipun demikian, SIBI tetap menjadi sistem isyarat yang diakui secara formal dan telah memiliki kamus resmi yang diterbitkan serta disebarluaskan ke sekolah-sekolah sejak tahun 2001, sehingga posisinya tetap kuat sebagai bahasa isyarat baku dalam konteks pendidikan formal di Indonesia hingga saat ini (Rachmawati, 2020).



Gambar 2. 6 Abjad SIBI (Yayasan Peduli Kasih ABK, 2023)

2.7 CRISP-DM

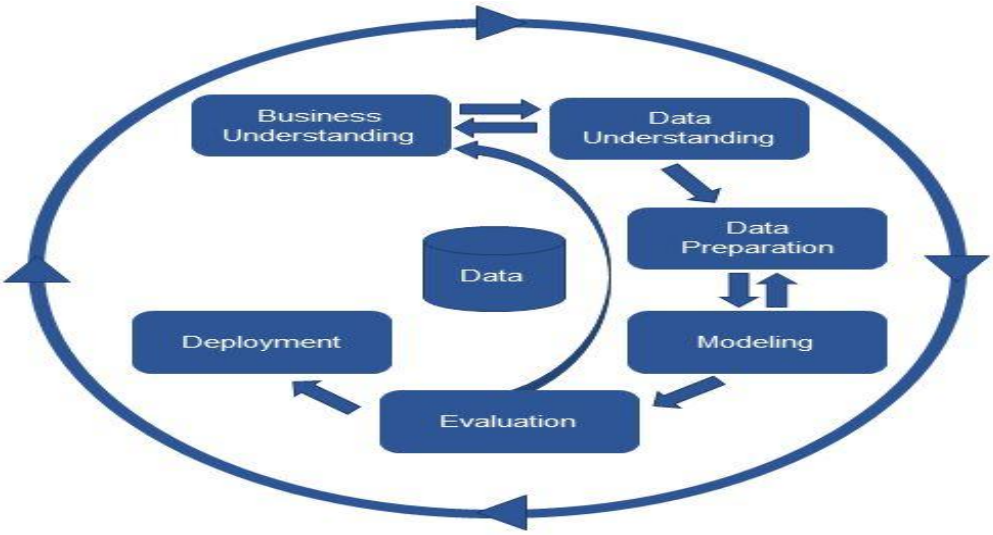
Cross Industry Standard Process for Data Mining (CRISP-DM) merupakan metodologi yang menyediakan pendekatan umum dan terstruktur untuk merencanakan proyek data mining, serta hingga saat ini masih menjadi metodologi paling populer di bidang data science, khususnya pada area data mining (Rianti et al., 2023). Metodologi ini terdiri atas enam tahapan, yaitu *business understanding*,

data understanding, *data preparation*, *modelling*, *evaluation*, dan *deployment*, di mana setiap tahapan memiliki fase yang berbeda dan dapat disesuaikan dengan kebutuhan proyek masing-masing peneliti (Rianti et al., 2023). Selain digunakan dalam bidang data mining, CRISP-DM juga dapat diterapkan pada berbagai area terkait seperti *artificial intelligence*, *machine learning*, *deep learning*, hingga *big data*, karena tahapannya yang fleksibel dan tidak terlalu kompleks sehingga mudah dipahami bahkan oleh peneliti pemula (Rianti et al., 2023).

Penerapan CRISP-DM dalam penelitian nyata dapat dilihat pada studi prediksi pendapatan menggunakan algoritma C4.5 di sebuah dinas perhubungan. Pada tahap *business understanding*, penelitian tersebut mendefinisikan tujuan bisnisnya sebagai pengembangan model prediksi pendapatan untuk mendukung pengambilan keputusan strategis, kemudian dilanjutkan dengan *data understanding* berupa pengumpulan dan eksplorasi data pendapatan individu (Ruswanti et al., 2024). Pada tahap *data preparation*, data dibersihkan dan diproses dengan memilih faktor-faktor penting seperti pendidikan, pekerjaan, dan status perkawinan, sebelum akhirnya masuk ke tahap *modelling* menggunakan algoritma C4.5 dan dievaluasi tingkat akurasinya (Ruswanti et al., 2024). Hasil penelitian tersebut menunjukkan bahwa model prediksi mampu mencapai akurasi sebesar 75%, dengan jenis kendaraan Light Truk diprediksi sebagai penyumbang pendapatan uji kir terbesar (Ruswanti et al., 2024).

Meskipun banyak digunakan, penerapan CRISP-DM tetap memiliki kelebihan sekaligus tantangan yang perlu dipertimbangkan oleh peneliti. Di antara kelebihannya adalah statusnya sebagai standar *de-facto* yang telah banyak diterima dalam proyek data mining, tahapannya yang jelas dan terstruktur sehingga mudah dipahami, serta adanya tahap *business understanding* di awal yang membantu menyelaraskan kebutuhan teknis dengan tujuan bisnis sejak dini (Rianti et al., 2023). Di sisi lain, tantangan yang sering muncul antara lain metodologi ini dianggap kurang mengalami pembaruan signifikan sejak pertama kali dirilis, prosesnya yang menyerupai model *waterfall* sehingga membutuhkan waktu cukup banyak pada tahap-tahap awal sebelum masuk ke *modelling*, serta belum adanya

kerangka komunikasi yang jelas antar-anggota tim dalam proyek berskala besar (Rianti et al., 2023).



Gambar 2. 7 CRISP-DM (Nurfatih, Nurmalasari dan Prakarsyah, 2024)

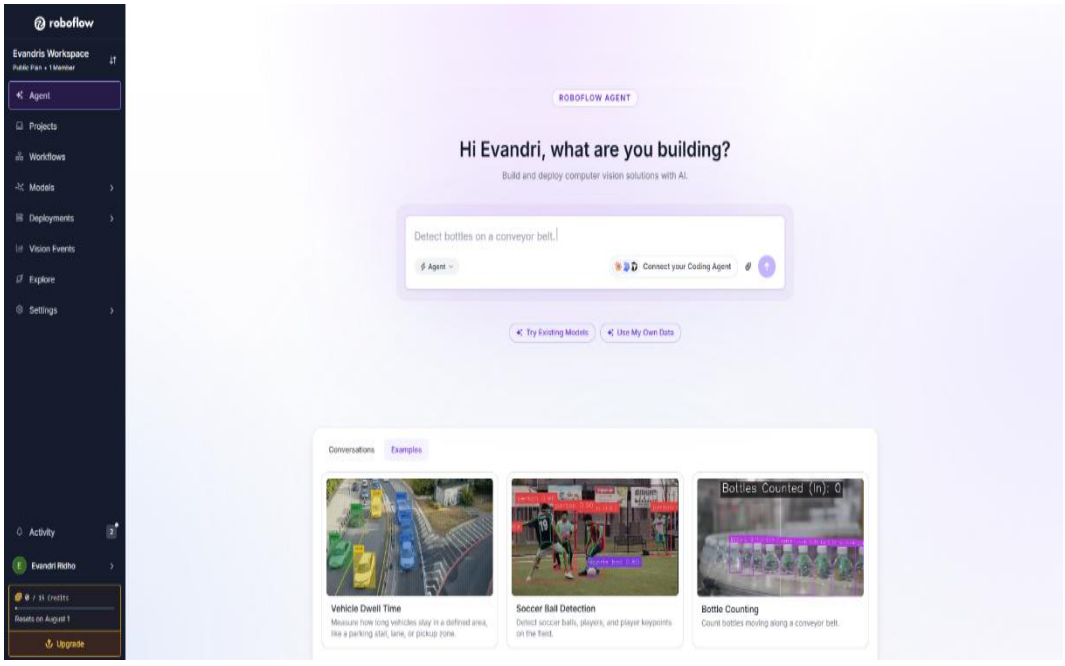
2.8 *Roboflow*

Roboflow merupakan sebuah platform berbasis web yang dirancang untuk mendukung proses pengembangan model computer vision, mulai dari pengumpulan data, anotasi, hingga teknik pelatihan model. Roboflow banyak dimanfaatkan oleh peneliti maupun praktisi karena menyediakan fitur manajemen dataset yang memudahkan proses anotasi data gambar sebelum digunakan untuk melatih model deteksi objek (Isma et al., 2025; Putra et al., 2024).

Secara fungsional, Roboflow menyediakan berbagai fitur yang mendukung keseluruhan alur kerja dalam pengembangan model deteksi objek. Pada tahap anotasi, setiap objek pada citra diberi bounding box dan label kelas secara manual menggunakan tool anotasi yang tersedia di platform tersebut. Roboflow juga menyediakan fitur pra-pemrosesan seperti auto-orient untuk menyeragamkan orientasi citra, penghapusan data yang tidak berlabel, pengubahan ukuran citra secara seragam, serta augmentasi data seperti flipping, cropping, dan rotasi untuk memperkaya variasi dataset pelatihan. Setelah proses tersebut selesai, dataset dapat diekspor dalam format yang kompatibel dengan model YOLO dan diunduh

langsung ke lingkungan pengembangan seperti Google Colab melalui API yang disediakan Roboflow (Isma et al., 2025; Putra et al., 2024).

Selain untuk anotasi dan pelatihan, Roboflow juga menyediakan fitur deploy yang memungkinkan model hasil pelatihan langsung diuji dan dipantau di dalam platform itu sendiri. Platform ini memungkinkan pengaturan ambang batas deteksi dan tingkat overlap, sehingga sensitivitas model dapat disesuaikan dengan kebutuhan penerapan di lapangan. Fitur ini terbukti bermanfaat pada penelitian deteksi penyakit daun durian menggunakan YOLOv8, di mana model yang telah dilatih dapat langsung diuji melalui antarmuka Roboflow dan menghasilkan deteksi dengan tingkat kepercayaan yang dapat disesuaikan sesuai kebutuhan (Putra et al., 2024).Hal ini menunjukkan bahwa Roboflow tidak hanya berperan pada tahap persiapan dataset, tetapi juga mendukung tahap evaluasi dan



Gambar 2. 8 Tampilan Beranda *Roboflow*

2.9 *Kaggle*

Kaggle merupakan sebuah platform daring yang dikembangkan untuk menunjang aktivitas para praktisi ilmu data dan pembelajaran mesin, di mana pengguna dapat mengunggah, membagikan, serta mengakses berbagai kumpulan data untuk keperluan penelitian maupun pengembangan model. Kaggle dikenal luas

sebagai salah satu platform terkemuka dalam bidang ilmu data, namun tetap menghadapi tantangan terkait integritas data, misalnya adanya praktik kecurangan seperti bot voting yang dapat memengaruhi keaslian kompetisi dan dataset yang diunggah oleh pengguna (Sudriyanto et al., 2024).

Selain berfungsi sebagai wadah kompetisi, Kaggle juga menyediakan berbagai kumpulan data publik dari beragam bidang, mulai dari kesehatan, ekonomi, hingga lingkungan, yang dapat diunduh secara bebas oleh peneliti maupun mahasiswa untuk mendukung proses analisis dan pemodelan. Ketersediaan dataset yang beragam dan mudah diakses inilah yang menjadikan Kaggle banyak dimanfaatkan sebagai sumber data sekunder dalam penelitian di bidang ilmu komputer dan kecerdasan buatan.

Sebagai contoh pemanfaatannya dalam penelitian akademik, Kaggle digunakan sebagai sumber Heart Disease Dataset yang terdiri atas 1.025 baris data dengan 14 atribut untuk membangun model klasifikasi penyakit jantung menggunakan algoritma K-Nearest Neighbor (K-NN), Logistic Regression, dan Decision Tree (Carolina Wibowo & Ardi Lestari, 2024). Hal ini menunjukkan bahwa dataset dari Kaggle tidak hanya digunakan untuk kompetisi, tetapi juga banyak dijadikan rujukan dalam penelitian ilmiah maupun tugas akhir mahasiswa di bidang machine learning dan data science.

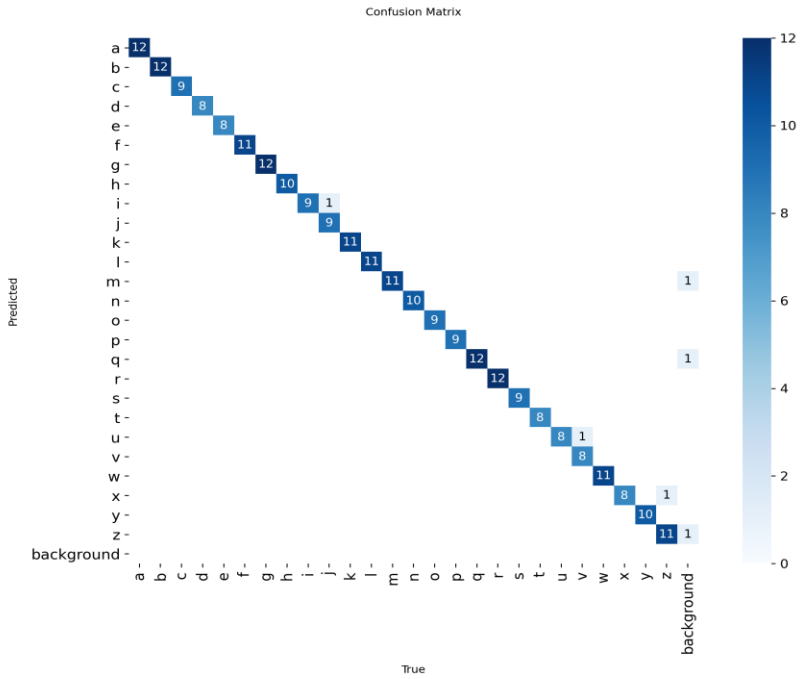


Gambar 2. 9 Logo *Kaggle*

2.10 Evaluasi Model

2.10.1 *Confusion Matrix*

Evaluasi kinerja model klasifikasi maupun deteksi objek umumnya dilakukan menggunakan *confusion matrix*, yaitu table yang membandingkan hasil prediksi model dengan nilai sebenarnya pada data pengujian. *Confusion matrix* terdiri dari atas empat komponen utama, yaitu True Positive (TP) untuk kasus ketika model memprediksi kelas positif dengan benar, True Negative (TN) untuk prediksi kelas negative yang benar, False Positive (FP) ketika model salah memprediksi kelas positif padahal sebenarnya negatif, serta False Negative (FN) ketika model salah memprediksi kelas negatif padahal sebenarnya positif (Hakim et al., 2025).



Gambar 2. 10 Confusion Matrix

2.10.2 Precision dan Recall

Berdasarkan hasil keempat komponen *confusion matrix*, dapat dihitung metrik *precision*, yaitu proporsi prediksi positif yang benar terhadap seluruh prediksi positif yang dihasilkan model, serta *recall*, yaitu proporsi kelas positif yang

berhasil diprediksi dengan benar dibandingkan seluruh kelas positif yang sebenarnya ada pada data (Hakim et al., 2025).

2.10.3 Mean Average Precision (mAP)

Selain precision dan recall, pada kasus deteksi objek dikenal pula metrik mean Average Precision (mAP) yang mengukur seberapa baik performa model dalam mendeteksi dan melokalisasi objek pada citra. Sebelum menghitung nilai mAP, terlebih dahulu dilakukan penyesuaian nilai threshold pada Intersection over Union (IoU) untuk menentukan validitas suatu objek yang terdeteksi, kemudian dibangun kurva Recall-Precision yang diinterpolasi untuk memperoleh nilai Average Precision (AP) pada setiap kelas (Ja'far & Udjulawa, 2025). Nilai mAP itu sendiri diperoleh dengan merata-ratakan seluruh nilai AP dari setiap kelas yang ada, sehingga mAP dapat memberikan gambaran menyeluruh mengenai akurasi deteksi model pada seluruh kategori objek yang diuji, bukan hanya pada satu kelas tertentu saja (Ja'far & Udjulawa, 2025).

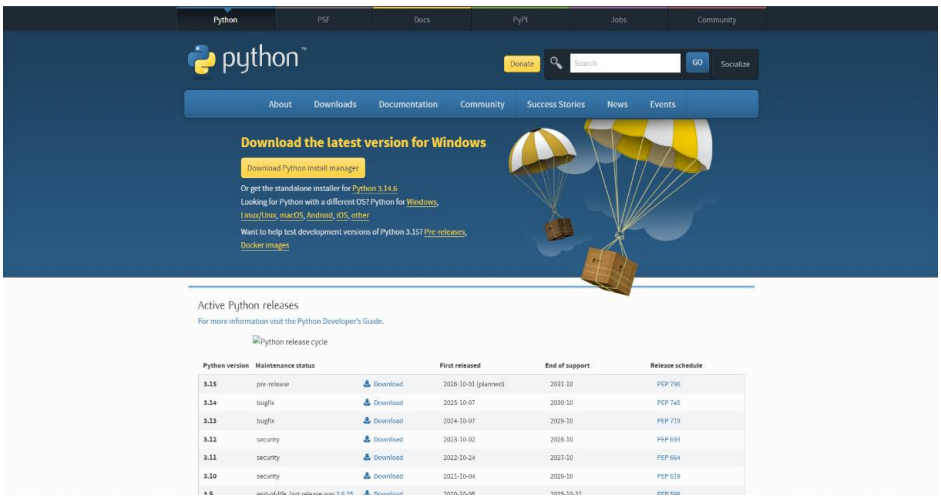
2.11 Python

Python merupakan bahasa pemrograman yang bersifat interpreted, interaktif, serta berorientasi objek. Bahasa ini pertama kali dirancang pada tahun 1990 oleh Guido van Rossum, dan seperti kebanyakan bahasa skrip lainnya, Python bersifat gratis bahkan untuk penggunaan komersial, serta dapat dijalankan di hampir semua komputer modern. Program Python secara otomatis dikompilasi menjadi bytecode yang bersifat platform-independent oleh kompiler, yang kemudian diinterpretasikan (Nandita & Yahfizham, 2023). Python juga dikenal sebagai bahasa pemrograman yang populer dan mudah dipelajari, sehingga banyak digunakan dalam pengembangan perangkat lunak, kecerdasan buatan, pengembangan web, machine learning, hingga analisis data (Ua et al., 2023).

Salah satu kekuatan utama Python terletak pada ekosistem library-nya yang sangat luas. Python menyediakan berbagai library, seperti NumPy untuk komputasi numerik, Pandas untuk analisis data, Matplotlib untuk visualisasi data, dan Scikit-learn untuk machine learning, yang masing-masing memudahkan penggunanya untuk menyelesaikan tugas tertentu dengan cepat dan efisien. Selain itu, Python

juga mudah diintegrasikan dengan teknologi lain seperti basis data, big data tools, dan framework web, sehingga memudahkan proses mengakses dan mengelola data dari berbagai sumber (Ua et al., 2023).

Di sisi lain, seperti bahasa pemrograman pada umumnya, Python juga memiliki kelebihan dan kekurangan yang perlu dipertimbangkan dalam penggunaannya. Beberapa kelebihan Python antara lain bersifat open-source, memiliki framework graphical user interface yang lengkap dan memudahkan pemula, kompatibel dengan sistem operasi Mac maupun Windows, serta didukung oleh komunitas pengembang yang terus berkembang. Meski demikian, Python juga memiliki beberapa kekurangan, seperti performa yang relatif lebih lambat karena merupakan bahasa yang ditafsirkan (interpreted), penggunaan memori yang cukup besar, serta kurang ideal untuk pengembangan aplikasi mobile (Nandita & Yahfizham, 2023).



Gambar 2. 11 Tampilan Beranda Python

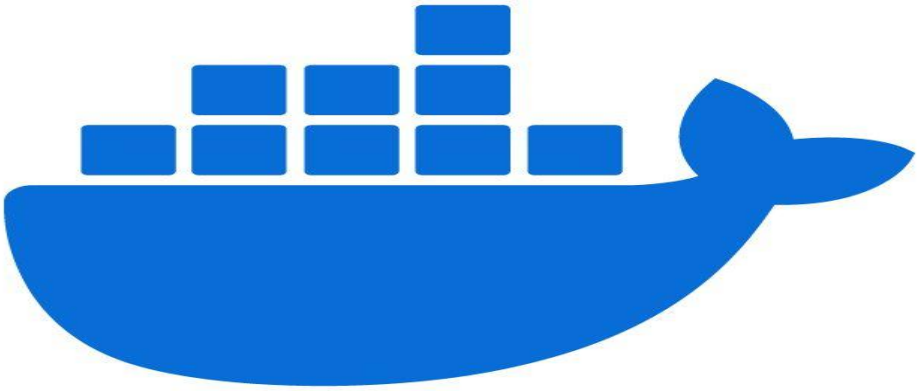
2.12 Docker

Docker merupakan salah satu solusi virtualisasi berbasis *container* yang dipilih untuk menggantikan pendekatan *virtual machine* (VM) konvensional karena keterbatasan perangkat keras dari sisi penyimpanan dan RAM. Berbeda dengan VM yang harus menjalankan *guest OS* secara terpisah dari *host* sehingga menyebabkan penggunaan sumber daya server menjadi berlebih, *Docker* memungkinkan setiap

container berbagi sistem operasi yang sama dengan *host*, sehingga menjadikannya jauh lebih ringan dan lebih cepat merespons ketika digunakan (Megantara et al., 2022). Karakteristik inilah yang membuat proses *booting container* dapat berlangsung hanya dalam hitungan detik, serta menjadikan biaya pengelolaan sistem berbasis *container* jauh lebih rendah dibandingkan mesin virtual.

Menurut Megantara et al. (2022) menerapkan *Docker* di atas *Host OS* Linux (CentOS) untuk membangun server yang menampung berbagai layanan seperti *Jupyter Notebook*, *Portainer*, dan *Nginx*, guna mendukung kegiatan pemrograman mahasiswa yang membutuhkan sumber daya komputasi besar, termasuk pemrograman berbasis *deep learning* dengan dukungan GPU melalui *NVIDIA-CUDA Toolkit*. Untuk mempermudah manajemen *container* secara visual, penelitian tersebut memanfaatkan *Portainer* sebagai antarmuka berbasis GUI, serta *Grafana* untuk memantau tingkat utilitas server seperti penggunaan CPU dan memori secara *real-time*.

Efektivitas implementasi *Docker* ini kemudian dievaluasi melalui survei terhadap 40 partisipan mahasiswa program doktoral yang menggunakan infrastruktur tersebut untuk kebutuhan penelitian dan penyelesaian disertasi. Hasil kuesioner pada penelitian tersebut menunjukkan tingkat kepuasan yang tinggi, di antaranya 90% partisipan menyatakan dapat mengakses server *Docker* tanpa kendala, serta 75% partisipan merasa sangat terbantu dengan adanya server tersebut dalam mendukung kegiatan penelitian mereka. Temuan ini menunjukkan bahwa penerapan *Docker* efektif dalam membantu institusi pendidikan menyediakan infrastruktur komputasi yang fleksibel tanpa memerlukan penambahan biaya perangkat keras maupun listrik.



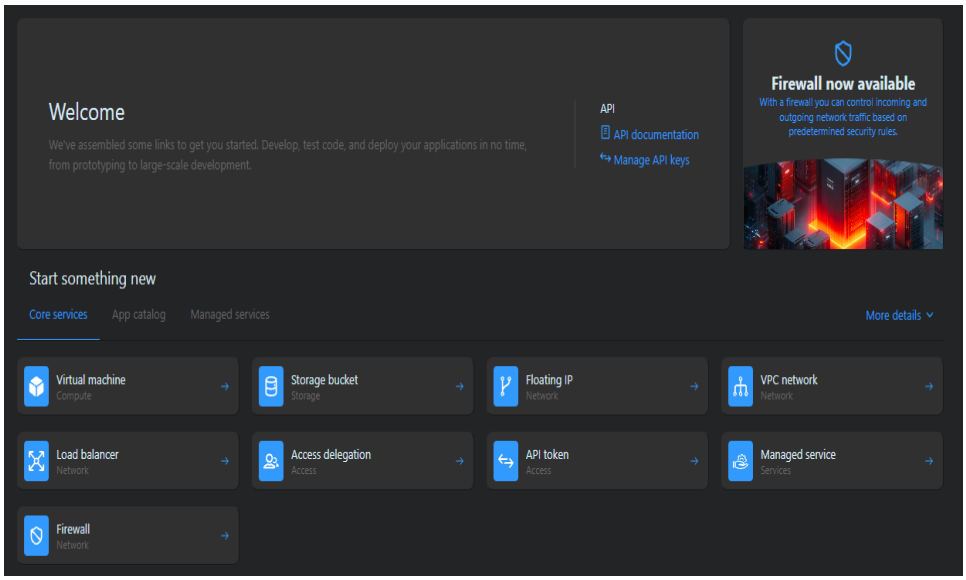
Gambar 2. 12 Logo Docker

2.13 *Virtual Privater Server*

IDCloudHost merupakan salah satu penyedia layanan *cloud hosting* lokal yang menyediakan solusi *Virtual Private Server* (VPS) untuk mendukung proses *deployment* aplikasi maupun sistem berbasis *container*. Layanan ini dipilih karena memberikan kontrol penuh terhadap *resource* server (CPU, RAM, penyimpanan) yang dialokasikan secara eksklusif untuk satu pengguna, sehingga performanya tidak terpengaruh oleh aktivitas pengguna lain dalam server fisik yang sama. (Gea et al., 2023) menjelaskan bahwa VPS merupakan server pribadi yang seluruh sumber dayanya digunakan oleh satu *user* saja dan tidak dipengaruhi oleh *user* lainnya, di mana teknologi virtualisasi memungkinkan satu server fisik dibagi menjadi beberapa *resource* server yang berjalan secara independen.

Penggunaan VPS pada penelitian ini juga didukung oleh hasil pengujian performa yang menunjukkan kelayakan teknologi ini untuk kebutuhan hosting maupun *deployment* aplikasi. Berdasarkan pengujian yang dilakukan (Gea et al., 2023), rerata beban penggunaan *resource* VPS tergolong sangat baik, dengan pemakaian CPU sekitar 2,9%, memori sekitar 71,23%, serta *disk space* sekitar 16,1%, sementara pengujian parameter *Quality of Service* (QoS) berupa *delay* dan *jitter* juga menunjukkan hasil yang tergolong bagus hingga sangat bagus. Temuan ini memperkuat alasan pemilihan VPS sebagai infrastruktur untuk men-*deploy container Docker* yang telah dibangun pada penelitian ini, karena stabilitas dan efisiensi *resource*-nya sudah teruji secara empiris.

Dengan memanfaatkan IDCloudHost sebagai penyedia layanan VPS, proses *deployment* sistem deteksi yang dikembangkan pada penelitian ini dapat berjalan secara stabil dan dapat diakses secara *online* tanpa bergantung pada infrastruktur lokal. Hal ini memberikan fleksibilitas bagi pengujian sistem, baik dari sisi pengembang maupun pengguna akhir, karena *server* dapat diakses kapan pun tanpa perlu mengandalkan perangkat keras milik pribadi yang harus menyala sepanjang waktu.



Gambar 2. 13 Dashboard IDCloud

2.14 Penelitian Terdahulu

Penelitian ini mengacu pada sejumlah studi terdahulu sebagai landasan dalam perancangan yang menggunakan metode *Cross-Industry Standard Process for Data Mining* (CRISP-DM). Beberapa referensi yang menjadi dasar pembelajaran tersebut dirangkum pada Tabel 2.1.

Tabel 2. 1 Tabel Penelitian Terdahulu

NO	Nama	Tahun	Judul	Penelitian
1	Darmatasia	2021	Pengenalan Sistem Isyarat Bahasa Indonesia (SIBI) Menggunakan Gradient-Convolutional	Menggunakan CNN dengan filter *gradient* pada lapisan *input* untuk mengenali 5 alfabet SIBI (A–E) dari 600 citra. Filter *gradient* meningkatkan akurasi dari 25% (tanpa filter)

NO	Nama	Tahun	Judul	Penelitian
			Neural Network	menjadi 85%, dengan akurasi terbaik 98% pada iterasi lebih tinggi. Diuji pula pada 3 *benchmark dataset* (Michal, Massey, Marcel) dan terbukti unggul dibanding metode CNN lain pada penelitian sejenis.
2	Suharjito, Narada Thiracitta, & Herman Gunawan	2021	SIBI Sign Language Recognition Using Convolutional Neural Network Combined with Transfer Learning and non-trainable Parameters	Menggunakan model *Inflated 3D* (I3D) CNN dikombinasikan *transfer learning* dari dataset ImageNet dan Kinetics untuk mengenali 10 kata isyarat SIBI dari 200 video (2 *signer*). Diuji 5 kombinasi *layer freezing*; model dengan *freeze* paling banyak (*Inflated 3D1-7*) memberikan hasil terbaik dengan akurasi validasi 100% dan akurasi pengujian 97,50%, presisi 98,00%, dan *recall* 97,50%.
3	Oky Dwi Nurhayati, Dania Eridani, & Muhammad Hafiz Tsalavin	2022	Sistem Isyarat Bahasa Indonesia (SIBI) Metode Convolutional Neural Network Sequential secara Real Time	Menggunakan CNN *sequential* untuk mengenali 29 kelas isyarat (26 alfabet + spasi, hapus, tidak terklasifikasi) dari 29.000 citra, dengan *pre-processing* HSV, *cropping*, dan *thresholding*. Pengujian pada kondisi redup, sedikit terang, terang, dan sangat terang.
4	Didik Tristianto & Michael Arthur Limantara	2024	Sistem Pembelajaran Isyarat Bahasa Indonesia (SIBI)	Membangun arsitektur CNN sendiri (2 lapisan konvolusi, 2 *pooling*, 2 *fully connected*) untuk mengenali 26 alfabet

NO	Nama	Tahun	Judul	Penelitian
			Menggunakan Metode Convolutional Neural Network (CNN)	SIBI dari 2.600 citra webcam (100 citra/kelas), dengan *pre-processing grayscale*, *Gaussian blur*, dan *adaptive threshold*. Hasil pelatihan mencapai akurasi ±99% dengan *loss* 3,7%. Sistem juga mampu menyusun huruf menjadi kata dan kalimat secara *real-time*, dengan akurasi pengujian 95,8% pada ruangan redup dan 97,23% pada ruangan terang.

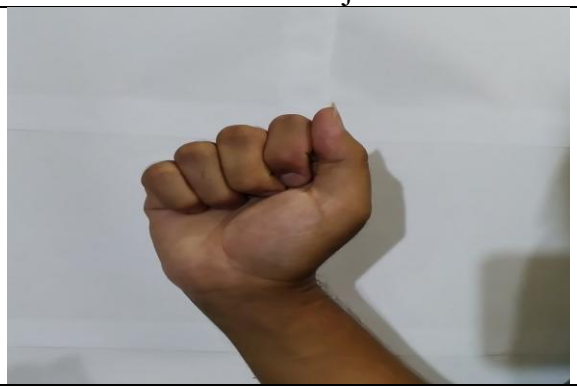
3. METODE PENELITIAN

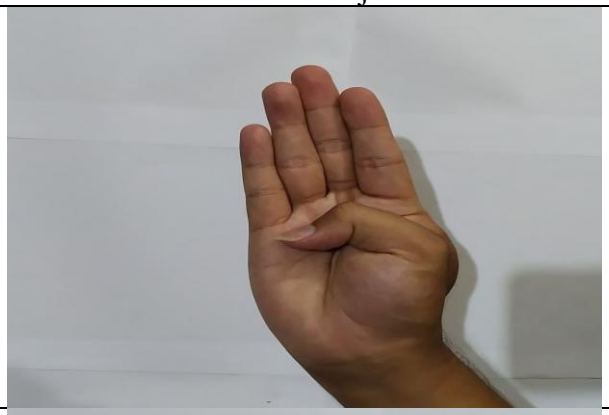
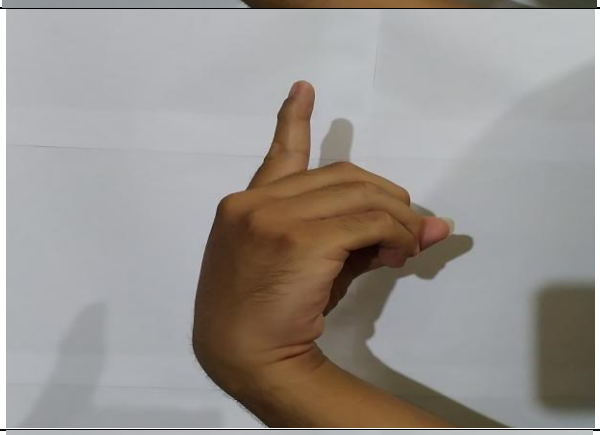
3.1 Tahapan Penelitian

Penelitian ini menggunakan metodologi CRISP-DM (Cross Industry Standard Process for Data Mining) sebagai kerangka kerja pengembangan sistem. Metodologi ini dipilih karena tahapannya jelas dan terstruktur, sehingga proses pengembangan model klasifikasi dapat dilakukan secara sistematis. Selain itu, CRISP-DM bersifat iteratif, artinya setiap tahapan dapat dilakukan secara berulang apabila ditemukan kekurangan pada tahap sebelumnya, sehingga model yang dihasilkan dapat terus disempurnakan hingga mencapai performa yang optimal sesuai dengan tujuan penelitian.

Objek yang digunakan dalam penelitian ini adalah abjad Sistem Isyarat Bahasa Indonesia (SIBI) yang direpresentasikan dalam bentuk citra gestur tangan. Setiap abjad memiliki bentuk gestur yang berbeda dan menjadi dasar dalam pembentukan dataset yang akan digunakan dalam proses klasifikasi. Berikut merupakan sampel abjad SIBI yang digunakan dalam penelitian ini:

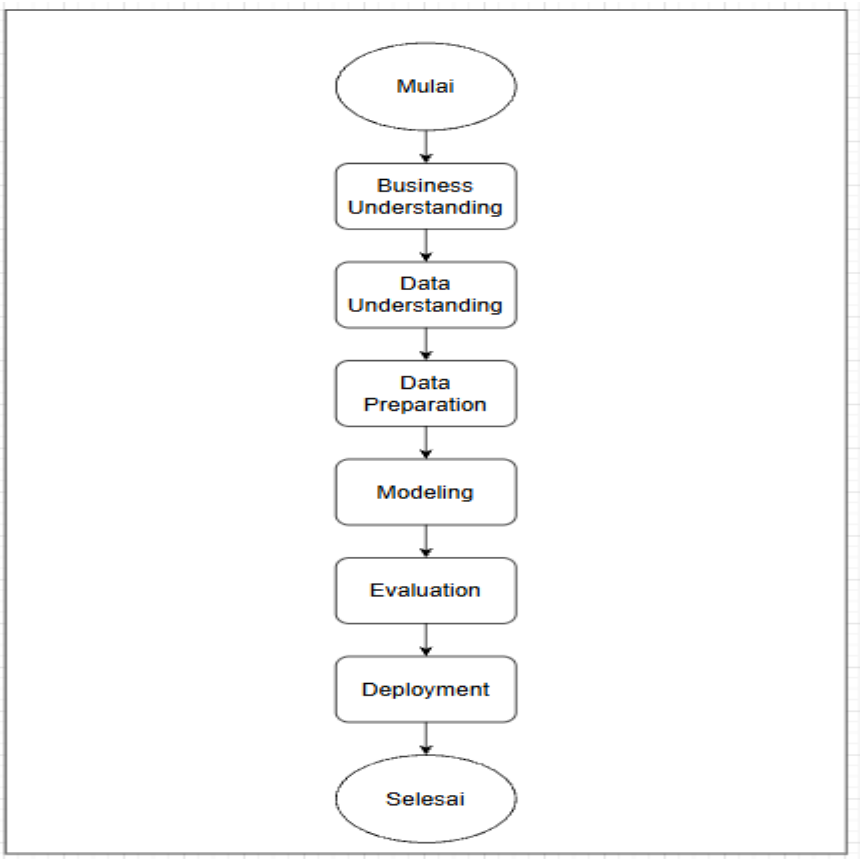
Tabel 3. 1 Sampel Abjad SIBI

NO	Gambar Abjad	Abjad
1		A

NO	Gambar Abjad	Abjad
2		B
3		C
4		D
5		E

Tabel 3.1 menampilkan beberapa sampel citra gestur tangan yang merepresentasikan abjad dalam SIBI, yang digunakan sebagai gambaran umum bentuk data yang akan diolah pada tahap selanjutnya. Sampel-sampel ini diambil dari beberapa kelas abjad yang berbeda untuk memberikan gambaran awal mengenai variasi bentuk tangan, sudut pengambilan gambar, serta karakteristik visual lain yang perlu diperhatikan sebelum data tersebut masuk ke tahap pengolahan lebih lanjut. Dengan melihat contoh-contoh ini, diharapkan pembaca dapat lebih mudah memahami jenis data yang menjadi dasar dalam pengembangan model deteksi pada penelitian ini.

Keseluruhan proses penelitian, mulai dari pemahaman kebutuhan hingga implementasi sistem, mengikuti tahapan CRISP-DM (*Cross-Industry Standard Process for Data Mining*) yang terdiri dari enam fase, yaitu *Business Understanding*, *Data Understanding*, *Data Preparation*, *Modeling*, *Evaluation*, dan *Deployment*. Setiap fase pada metodologi ini saling berkaitan dan dilakukan secara berurutan namun tetap fleksibel, sehingga apabila ditemukan kekurangan pada satu tahap, penulis dapat kembali ke tahap sebelumnya untuk melakukan penyesuaian sebelum melanjutkan ke tahap berikutnya. Pendekatan ini dipilih karena dinilai sesuai dengan karakteristik penelitian yang berbasis pengolahan data dan pengembangan model *machine learning*. Tahapan tersebut digambarkan pada diagram berikut:



Gambar 3. 1 Diagram CRISP-DM

3.1.1 *Business Understanding*

Tahapan penelitian yang dilakukan mengacu pada metodologi CRISP-DM (*Cross-Industry Standard Process for Data Mining*) sebagaimana ditampilkan pada Gambar 3.1. Tahap *Business Understanding* pada penelitian ini difokuskan pada pemahaman masalah kesenjangan komunikasi antara penyandang tunarungu pengguna Sistem Isyarat Bahasa Indonesia (SIBI) dan masyarakat umum. Meskipun SIBI merupakan standar komunikasi resmi, penggunaannya di sektor publik masih sangat terbatas (Suharjito et al., 2021), dan upaya pembelajaran manual dirasa kurang efektif karena memakan waktu lama. Berdasarkan data Kementerian Kesehatan RI (2023), prevalensi gangguan pendengaran di Indonesia memerlukan solusi teknologi untuk menjembatani komunikasi ini secara lebih inklusif.

Beberapa penelitian sebelumnya telah berupaya menjawab tantangan ini melalui pendekatan *computer vision* berbasis *Convolutional Neural Network* (CNN) untuk mendeteksi abjad SIBI (Darmatasia, 2021; Dwi Nurhayati et al., 2021; Tristianto & Limantara, 2024). Belum banyak penelitian yang mengimplementasikan model tersebut ke dalam platform antarmuka web yang dapat diakses dengan mudah dan seketika oleh masyarakat luas.

Merespons permasalahan tersebut, penelitian ini mengusulkan pengembangan aplikasi interaktif berbasis web menggunakan algoritma *You Only Look Once* versi 8 (YOLOv8) untuk mendeteksi abjad SIBI dari citra tangan statis. Sistem yang dibangun diharapkan tidak hanya berfokus pada aspek teknis AI, tetapi juga dapat diimplementasikan sebagai *assistive technology* yang mendukung komunikasi dua arah dalam masyarakat.

3.1.1.1 Kebutuhan Fungsional

Kebutuhan fungsional merupakan gambaran spesifikasi layanan, fitur, atau kemampuan teknis yang wajib disediakan oleh sistem agar tujuan utama dari penelitian dapat tercapai. Pada pengembangan sistem deteksi abjad SIBI berbasis *web* ini, kebutuhan tersebut berfokus pada interaksi inti antara pengguna, kamera, dan antarmuka aplikasi. Berdasarkan rancangan sistem yang dibangun, kebutuhan fungsional dari sistem ini diuraikan ke dalam poin-poin berikut:

1. Sistem harus mampu menangkap input berupa citra tangan pengguna melalui kamera perangkat.
2. Sistem dapat memproses citra masukan menggunakan model deteksi objek YOLOv8 untuk mengenali huruf SIBI (A-Z) yang diperagakan.
3. Sistem mampu menampilkan hasil deteksi pada antarmuka *web* berupa deteksi huruf statis SIBI beserta tingkat keyakinan dari prediksi model tersebut.
4. Sistem harus menyediakan halaman informasi kamus abjad SIBI (A-Z) dan menyediakan navigasi yang memungkinkan pengguna berpindah antara halaman deteksi dan halaman informasi dengan mudah.

3.1.1.2 Kebutuhan Non Fungsional

Kebutuhan nonfungsional mendefinisikan kriteria kualitas, performa, dan karakteristik operasional yang memastikan sistem dapat berjalan secara optimal, aman, dan nyaman digunakan oleh masyarakat umum. Aspek ini melengkapi kebutuhan fungsional dengan menitikberatkan pada *bagaimana* sistem tersebut beroperasi. erikut adalah rincian kebutuhan nonfungsional dari sistem yang dikembangkan:

1. Luaran : Sistem menampilkan hasil prediksi huruf SIBI beserta nilai *confidence* apabila objek tangan terdeteksi.
2. Ketahanan : Sistem harus dapat beroperasi dengan baik pada kondisi pencahayaan dalam ruangan standar dengan berbagai variasi latar belakang menggunakan kamera bawaan perangkat.
3. Kebergunaan : Antarmuka pengguna dirancang secara intuitif dan sederhana agar mudah dioperasikan oleh pengguna nonteknis sekalipun.
4. Keamanan : Sistem tidak menyimpan data citra wajah atau tangan pengguna maupun hasil prediksi ke dalam *database.*
5. Portabilitas : Aplikasi bersifat fleksibel untuk dijalankan melalui berbagai peramban dari berbagai jenis perangkat.

3.1.2 *Data Understanding*

Pada tahap *Data Understanding*, penelitian berfokus pada pemahaman terhadap dataset yang menjadi fondasi keberhasilan sistem pengenalan alfabet Sistem Isyarat Bahasa Indonesia (SIBI). Data yang digunakan merupakan data sekunder bernama "Datasets SIBI Sign Languange Alphabets" yang diperoleh dari repositori Kaggle. Dataset ini berisi sekumpulan citra tangan statis yang merepresentasikan 26 huruf alfabet (A–Z) dalam SIBI dengan berbagai variasi posisi tangan, sudut pandang, serta kondisi pencahayaan. Pengambilan citra juga dilakukan dengan latar belakang yang bervariasi, mulai dari latar polos hingga lingkungan nyata, guna memberikan keragaman yang merepresentasikan situasi penggunaan sistem di dunia nyata.

Dataset awal yang dikumpulkan terdiri dari 1.307 citra yang terdistribusi ke dalam 26 kelas label huruf. Untuk menjaga keseimbangan dan objektivitas evaluasi model, dataset awal tersebut terlebih dahulu di-*rebalance* menggunakan fitur pada platform Roboflow sehingga proporsi data terbagi menjadi 70% sebagai data latih, 20% sebagai data validasi, dan 10% sebagai data uji. Setelah proses pembagian selesai, teknik augmentasi menggunakan platform Roboflow diterapkan secara eksklusif hanya pada data latih untuk memperbanyak variasi data dan mencegah *overfitting*, sehingga jumlah data latih meningkat secara signifikan menjadi 3.137 citra.

Berdasarkan hasil *Exploratory Data Analysis* (EDA) dan pemeriksaan kualitas awal, distribusi kelas pada dataset terpantau relatif seimbang dan sebagian besar citra menampilkan objek tangan secara utuh tanpa *noise* yang mengganggu. Guna mengoptimalkan kualitas data sebelum masuk ke tahap pelatihan, diterapkan langkah *preprocessing* berupa *Auto-Orient* untuk memperbaiki orientasi gambar secara otomatis, serta *Resize* dengan metode *Fit (black edges)* untuk menyamakan dimensi citra menjadi 640x640 piksel. Dengan pemahaman dan pemrosesan data yang sistematis ini, model YOLOv8 yang dibangun diharapkan dapat mencapai kinerja deteksi yang optimal dan mampu mengenali data baru dengan baik pada pengujian sebenarnya.

3.1.3 *Data Preparation*

Tahap *Data Preparation* bertujuan untuk mempersiapkan dataset Sistem Isyarat Bahasa Indonesia (SIBI) agar siap digunakan dalam proses pelatihan model *Object Detection* YOLOv8. Tahap ini mencakup serangkaian langkah yang bertujuan untuk memastikan kualitas, konsistensi, serta representativitas data agar sesuai dengan kebutuhan model. Adapun langkah-langkah yang dilakukan pada tahap ini meliputi pengumpulan dan pemilihan citra tangan yang merepresentasikan setiap abjad SIBI, pelabelan (*annotation*) objek pada setiap citra sesuai dengan kelas abjad yang bersangkutan, serta penyesuaian format data agar kompatibel dengan arsitektur YOLOv8. Selain itu, dilakukan pula proses pembagian dataset menjadi data latih (*training*), data validasi (*validation*), dan data uji (*testing*) dengan

proporsi tertentu, guna memastikan model dapat dievaluasi secara objektif. Tahapan ini merupakan bagian penting dalam metodologi CRISP-DM, karena kualitas data yang dipersiapkan pada tahap ini akan sangat memengaruhi performa dan akurasi model pada tahap pelatihan (*modeling*) selanjutnya.

Dataset yang digunakan dalam penelitian ini diperoleh dari repositori Kaggle dengan judul "Datasets SIBI Sign Language Alphabets" yang dipublikasikan oleh M. Lanang Afkaar pada tahun 2021. Pada tahap awal sebelum pemrosesan, *dataset* ini memiliki total 1.307 citra SIBI. Data tersebut diunduh dalam bentuk kompresi berekstensi .zip. Setelah proses ekstraksi dilakukan, data secara otomatis terstruktur ke dalam tiga folder utama, yaitu *train*, *validation*, dan *test*, di mana keseluruhan citra di dalamnya menggunakan format ekstensi .jpg. Pemeriksaan integritas data juga dilakukan pada setiap folder untuk memastikan tidak ada *file* yang rusak atau hilang sebelum masuk ke tahap pengolahan lebih lanjut.

Selanjutnya, guna meningkatkan variasi data serta mengurangi potensi ketidakseimbangan kelas, diterapkan strategi augmentasi data. Teknik augmentasi yang diaplikasikan pada citra dalam penelitian ini meliputi pemotongan, pemutaran, penyesuaian tingkat kecerahan, pengaburan, serta penambahan derau. Setelah serangkaian proses augmentasi dan persiapan data tersebut selesai dilakukan, jumlah keseluruhan *dataset* mengalami peningkatan menjadi 3.137 citra SIBI, dengan distribusi data yang tetap terjaga untuk mengoptimalkan kinerja model pada saat tahap pelatihan. Langkah yang diterapkan meliputi:

1. *Data Annotate*

   Proses memberi label, tanda, atau penjelasan pada data abjad SIBI agar dapat dipahami dan digunakan untuk melatih sistem kecerdasan buatan.
2. *Data Splitting*

   Dataset abjad SIBI dibagi menjadi tiga *subset* yang terdiri dari 70% data *training*, 20% data *validation*, dan 10% data *testing*. Proporsi pembagian ini dipertahankan pada setiap *subset* untuk memastikan agar

distribusi kelas tetap seimbang, sehingga hasil evaluasi akhir model menjadi lebih representatif.

3. *Preprocessing*
    - *Auto-Orient*
    - *Resize*
4. *Data Augmentation*
    - *Crop*
    - *Rotation*
    - *Brightness*
    - *Blur*
    - *Noise*

3.1.4 *Modeling*

Tahap selanjutnya pada metode *CRISP-DM* adalah *modeling*, yaitu proses pelatihan model berbasis data yang telah dipersiapkan pada tahap sebelumnya. Proses ini dilakukan secara lokal menggunakan Antigravity, dengan spesifikasi perangkat keras berupa *GPU NVIDIA RTX 3050* (4 GB *VRAM*), RAM sistem sebesar 15,3 GB, dan penyimpanan sebesar 928,5 GB.

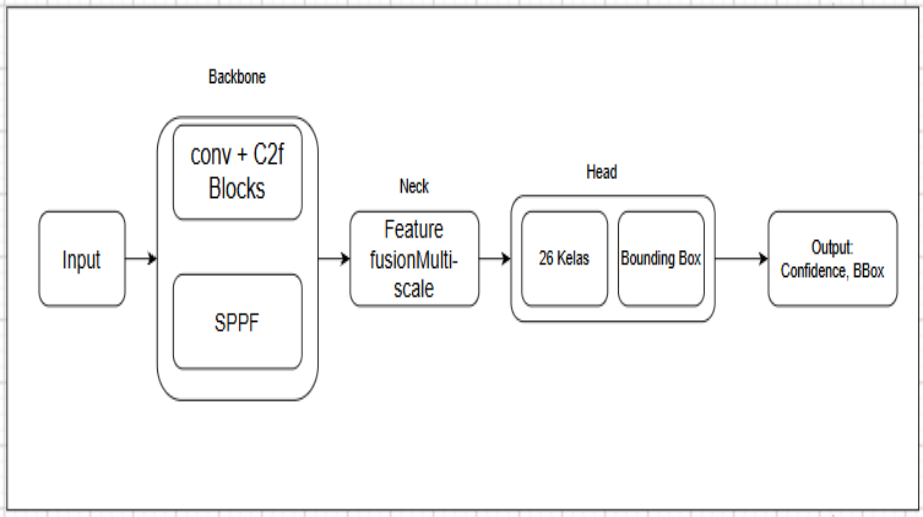
Sebelum memulai pelatihan, dilakukan instalasi pustaka *ultralytics* untuk mendukung implementasi arsitektur *YOLOv8*. Dataset SIBI (Sistem Isyarat Bahasa Indonesia) yang digunakan telah disiapkan secara lokal dalam format anotasi YOLOv8, mencakup data citra (.jpg) yang telah dibagi ke dalam *split train, val,* dan *test*, beserta label (.txt) dan berkas konfigurasi *.yaml*.

Pada penelitian ini digunakan varian model *YOLOv8n*, yang dipilih karena ukurannya ringan dan sesuai untuk dijalankan pada perangkat dengan kapasitas *VRAM* terbatas, namun tetap mampu memberikan performa deteksi yang memadai. Parameter utama pelatihan yang digunakan adalah sebagai berikut:

- Epochs : 100
- Batch Size : mengikuti nilai *default* yang disesuaikan secara otomatis oleh sistem berdasarkan kapasitas *GPU* yang tersedia.

- Image Size : 640 × 640 piksel.
- Learning Rate : menggunakan nilai *default* bawaan *YOLOv8*.
- Plots : menggunakan pengaturan *default* dari *ultralytics* untuk memantau hasil pelatihan.

Proses pelatihan dijalankan dengan perintah *YOLO train*, dan selama proses berlangsung sistem secara otomatis menghasilkan berkas model (.pt) yang siap digunakan pada tahap *deployment*. Selain itu, dihasilkan pula metrik pemantauan berupa *Precision*, *Recall*, *mAP@0.5*, *mAP@0.5:0.95*, *Loss* (*box loss, class loss,* dan *DFL loss*), serta *Confusion Matrix* yang menggambarkan distribusi prediksi benar dan salah antar kelas huruf *SIBI*.



Gambar 3. 2 Aristektur YOLOv8 Penelitian

Arsitektur model YOLOv8 yang digunakan dalam penelitian ini terdiri dari tiga komponen utama, yaitu *backbone*, *neck*, dan *head*, sebagaimana ditampilkan pada Gambar 3.2. Tahap *backbone* berfungsi untuk melakukan ekstraksi fitur dari citra masukan menggunakan kombinasi *Conv*, *C2f Blocks*, dan *SPPF* (*Spatial Pyramid Pooling Fast*). Fitur yang dihasilkan kemudian diproses pada tahap *neck* melalui mekanisme *multi-scale feature fusion* untuk menggabungkan informasi fitur dari berbagai skala, baik kecil, sedang, maupun besar. Selanjutnya, tahap *head* melakukan prediksi terhadap 26 kelas huruf (A-Z) beserta lokasi *bounding box* dari objek yang terdeteksi. Hasil prediksi tersebut kemudian disaring menggunakan

Non-Maximum Suppression (NMS) untuk menghilangkan deteksi duplikat dengan *threshold confidence* sebesar ≥ 0,70, sehingga menghasilkan keluaran akhir berupa nilai *confidence* dan koordinat *bounding box* dari huruf yang terdeteksi.

3.1.5 *Evaluation*

Tahap *evaluation* dalam metode CRISP-DM merupakan fase yang penting untuk menilai kualitas serta kinerja *model* yang telah dilatih, sekaligus mengukur sejauh mana sistem yang dibangun mampu mencapai tujuan penelitian. Pada tahap ini, hasil dari proses *modeling* ditinjau kembali kesesuaiannya dengan permasalahan yang telah dirumuskan sebelumnya, untuk memastikan *model* yang dibangun layak digunakan.

Penilaian pada tahap ini menggunakan sejumlah metrik, yaitu *precision*, *recall*, *f1-score*, *support*, serta *accuracy* yang mencakup nilai *macro average* dan *weighted average*. Setiap metrik tersebut memiliki peran masing-masing dalam menggambarkan performa *model* secara lebih spesifik, misalnya *precision* yang menunjukkan ketepatan prediksi *model*, *recall* yang menunjukkan kemampuan *model* dalam mengenali seluruh data yang relevan, hingga *f1-score* yang merepresentasikan keseimbangan antara keduanya. Metrik-metrik tersebut digunakan secara bersamaan untuk memberikan gambaran menyeluruh mengenai performa *model*, sehingga hasil evaluasi yang diperoleh dapat lebih akurat, representatif, dan tidak hanya bergantung pada satu ukuran performa saja. Penjelasan lebih rinci mengenai masing-masing metrik dapat dilihat pada Tabel 3.2

Tabel 3. 2 Penjelasan Metrik Evaluasi

NO	Metrik	Fungsi
1	*Precision*	Menunjukkan tingkat ketepatan model saat memprediksi kelas positif, yaitu seberapa besar proporsi prediksi positif yang benar-benar sesuai dengan kondisi sebenarnya.
2	*Recall*	Menunjukkan kemampuan model dalam mengenali seluruh data positif yang seharusnya terdeteksi, yaitu seberapa banyak data positif yang berhasil ditemukan oleh model dari keseluruhan data positif yang ada.

NO	Metrik	Fungsi
3	*Confusion Matrix*	Merupakan representasi visual yang menunjukkan perbandingan antara label sebenarnya dengan hasil prediksi model klasifikasi. Prediksi yang tepat digambarkan melalui nilai True Positive (TP) dan True Negative (TN), sementara prediksi yang keliru direpresentasikan melalui nilai False Positive (FP) dan False Negative (FN).
4	mAP50	Merupakan nilai rata-rata dari average precision (AP) di seluruh kelas yang dihitung berdasarkan ambang batas IoU sebesar 0,5.
5	mAP50-95	Merupakan nilai rata-rata AP yang dihitung pada rentang ambang batas IoU mulai dari 0,5 hingga 0,95, dengan interval kenaikan sebesar 0,05 (0,5; 0,55; 0,6; ...; 0,95).

3.1.6 *Deployment*

Tahap *deployment* pada kerangka kerja *CRISP-DM* berfungsi untuk mengintegrasikan model deteksi huruf SIBI yang telah melalui proses pelatihan dan evaluasi ke dalam sebuah sistem yang dapat diakses dan digunakan secara langsung oleh pengguna. Pada penelitian ini, proses *deployment* dibangun menggunakan *framework FastAPI* yang dikembangkan pada lingkungan Antigravity, dengan memanfaatkan model YOLOv8n terbaik yang dihasilkan dari tahap sebelumnya.

Sistem pada penelitian ini dirancang untuk bekerja secara dengan menerima input langsung berupa aliran *frame* video dari kamera pengguna. Setiap *frame* yang diterima melalui *endpoint* tersebut langsung diinferensi oleh model YOLOv8n untuk mendeteksi huruf SIBI yang muncul pada tampilan kamera. Hasil deteksi kemudian dikirimkan kembali ke sisi pengguna berupa huruf yang berhasil ditebak beserta nilai *confidence*, yaitu tingkat keyakinan model terhadap prediksi tersebut, sehingga pengguna dapat mengetahui seberapa andal hasil deteksi yang ditampilkan pada setiap *frame*. Struktur sistem pada tahap *deployment* ini terdiri atas beberapa bagian utama sebagai berikut:

- Endpoint Deteksi – Membuat sebuah *endpoint* yang berfungsi menerima aliran *frame* video dari sisi pengguna untuk selanjutnya diproses oleh model.
- Proses Inferensi - Setiap *frame* yang diterima melalui *endpoint* tersebut langsung diinferensi oleh model YOLOv8n guna mendeteksi huruf SIBI yang muncul pada tampilan kamera.
- Pengiriman Hasil Deteksi - Sistem mengirimkan kembali hasil prediksi berupa huruf SIBI yang berhasil dikenali beserta nilai *confidence*, yaitu tingkat keyakinan model terhadap tebakan huruf tersebut, ke antarmuka pengguna secara langsung.
- Antarmuka Pengguna - Tampilan antarmuka dibangun menggunakan ReactJs yang dipadukan dengan TailwindCSS, sehingga pengguna dapat melihat hasil deteksi huruf beserta tingkat keyakinannya secara langsung dan interaktif melalui *browser* tanpa memerlukan proses instalasi tambahan.

4. HASIL DAN PEMBAHASAN

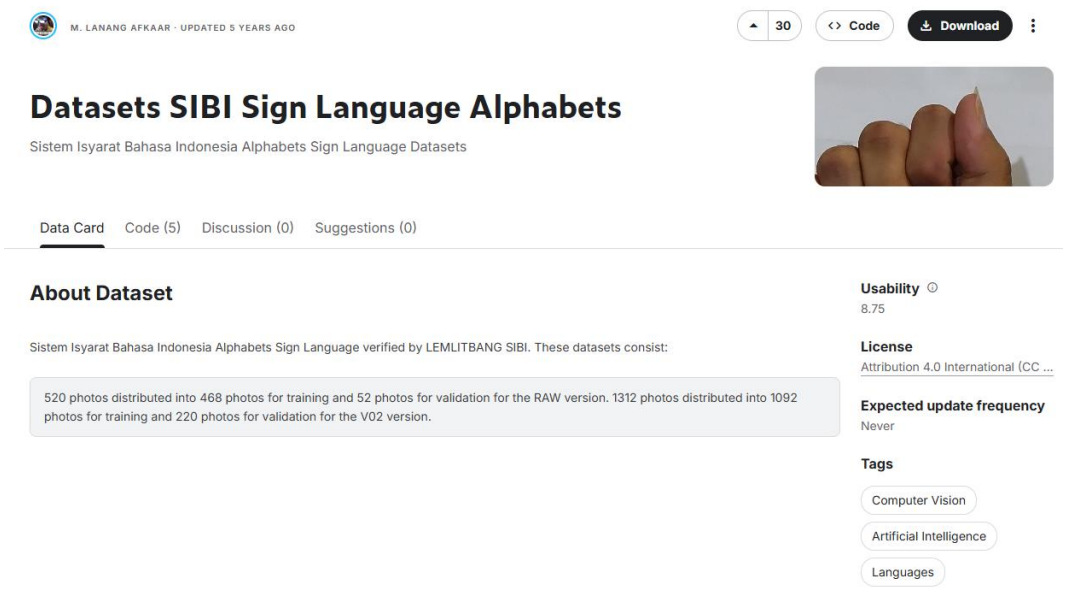
4.1 Hasil *Business Understanding*

Salah satu tantangan yang masih dihadapi penyandang tunarungu hingga saat ini adalah minimnya media komunikasi yang praktis dan mudah diakses ketika mereka harus berinteraksi dengan masyarakat umum. Hal ini tidak lepas dari kenyataan bahwa pemahaman masyarakat terhadap Sistem Isyarat Bahasa Indonesia (SIBI) masih sangat terbatas, sementara mempelajarinya secara konvensional membutuhkan waktu yang tidak singkat. Akibatnya, komunikasi antara penyandang tunarungu dan masyarakat umum kerap kali terhambat, terutama dalam situasi yang menuntut respons cepat, misalnya saat mengakses layanan publik yang membutuhkan interaksi langsung dan efisien.

Melihat kondisi tersebut, penelitian ini digagas dengan tujuan menghadirkan sebuah sistem yang mampu menerjemahkan abjad SIBI secara otomatis. Dengan adanya sistem ini, masyarakat umum yang belum memahami bahasa isyarat tetap dapat berkomunikasi dengan penyandang tunarungu tanpa perlu mempelajari SIBI terlebih dahulu. Pada akhirnya, sistem ini diharapkan dapat menjembatani kesenjangan komunikasi tersebut dan mendorong terciptanya interaksi sosial yang lebih inklusif serta efisien bagi kedua belah pihak.

4.2 Hasil *Data Understanding*

Tahap ini dilakukan untuk mengenali dan memahami karakteristik data yang akan digunakan dalam penelitian, mencakup proses pencarian, pengumpulan, serta penelusuran sumber data yang relevan dan dapat dipertanggungjawabkan keabsahannya. Data yang dikumpulkan berupa citra tangan yang merepresentasikan abjad Sistem Isyarat Bahasa Indonesia.



Gambar 4. 1 Dataset Abjad

Pada Gambar 4.1 menampilkan dataset abjad SIBI yang diperoleh dari platform Kaggle (https://www.kaggle.com/datasets/mlanangafkaar/datasets-lemlitbang-sibi-alphabets), yang digunakan sebagai sumber data utama dalam penelitian ini. Dataset tersebut terdiri dari 26 kelas, merepresentasikan masing-masing abjad dalam SIBI mulai dari A hingga Z. Setiap kelas berisi kumpulan citra gestur tangan yang merepresentasikan bentuk isyarat dari abjad yang bersangkutan, sehingga dapat digunakan sebagai acuan visual dalam proses pelatihan model deteksi objek pada penelitian ini.

Dataset ini dipilih karena tersedia secara publik dan dapat diakses secara terbuka melalui platform Kaggle, sehingga memudahkan proses pengumpulan data tanpa perlu melakukan pengambilan gambar secara manual untuk seluruh kelas abjad. Dengan cakupan 26 kelas yang mewakili keseluruhan abjad dalam SIBI, dataset ini dinilai cukup representatif untuk mendukung proses pelatihan model dalam mengenali variasi bentuk tangan pada setiap huruf.



Gambar 4. 2 Hasil Dataset Teranotasi

Pada Gambar 4.2 menampilkan hasil dataset yang telah melalui proses anotasi dan pelabelan secara manual menggunakan platform Roboflow, dengan jumlah keseluruhan sebanyak 1.307 data gambar yang telah diberi label. Adapun sumber data gambar yang digunakan dalam proses anotasi tersebut diperoleh dari dataset yang tersedia pada platform Kaggle.

4.3 Hasil *Data Preparation*

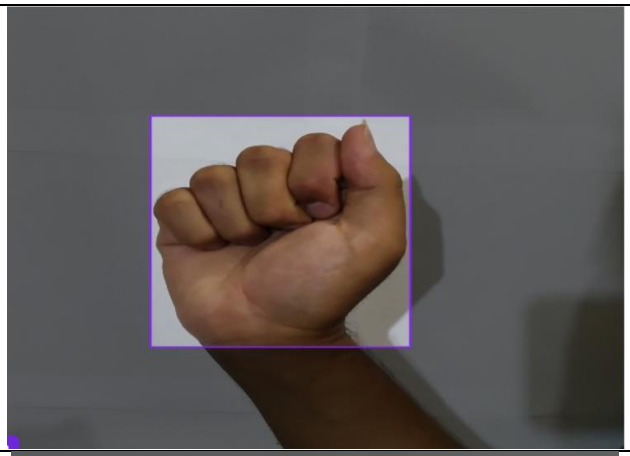
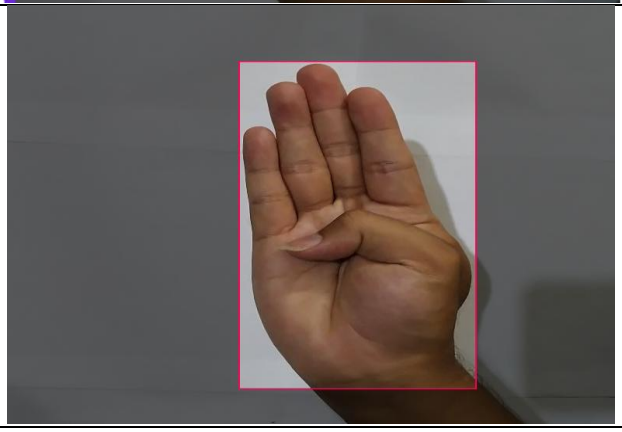
Data Preparation dilakukan untuk mengolah data mentah menjadi data yang siap digunakan dalam proses pelatihan model. Tahap ini diawali dengan proses anotasi, yaitu pemberian label pada setiap citra tangan sesuai dengan kelas abjad SIBI yang direpresentasikan menggunakan platform *Roboflow*. Setelah proses anotasi selesai, data kemudian dibagi ke dalam data latih, data validasi, dan data uji dengan proporsi tertentu untuk memastikan model dapat dievaluasi secara objektif.

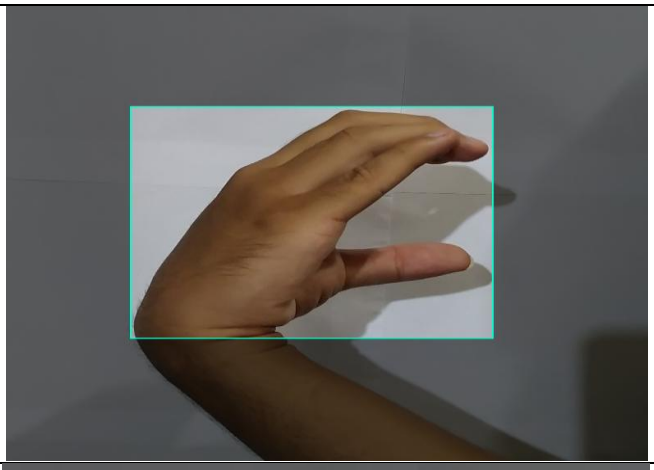
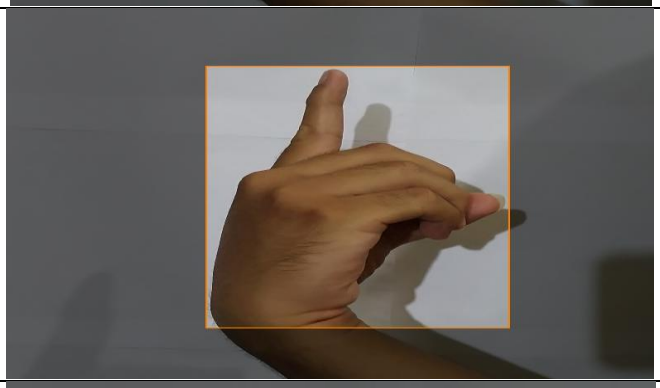
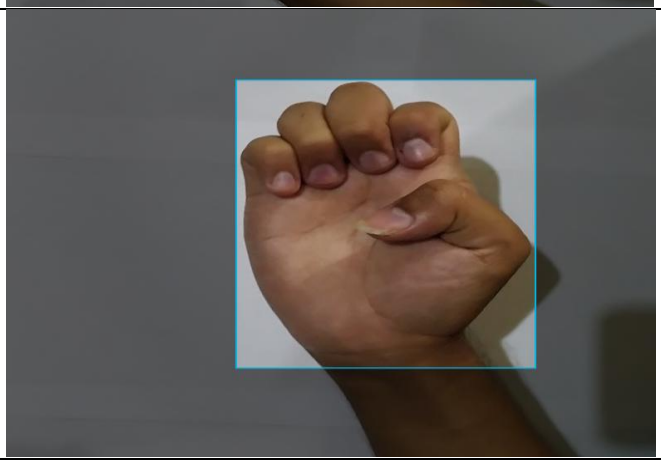
Selanjutnya, dilakukan proses *preprocessing* seperti penyesuaian ukuran dan normalisasi citra agar sesuai dengan format input yang dibutuhkan oleh model YOLOv8. Terakhir, dilakukan proses augmentasi data guna memperbanyak variasi citra pelatihan, seperti *rotasi*, *flipping*, atau perubahan pencahayaan, sehingga model memiliki kemampuan generalisasi yang lebih baik dalam mengenali variasi bentuk tangan pada setiap huruf.

4.3.1 Hasil *Anotate*

Pada tahap anotasi, data melalui proses pelabelan menggunakan teknik bounding box, yaitu dengan menempatkan kotak anotasi yang mengelilingi area gestur tangan pada setiap citra untuk menandai lokasi objek yang akan dideteksi oleh model.

Tabel 4. 1 Sampel Hasil *Anotate*

NO	Gambar Hasil *Anotate*	Abjad
1		A
2		B

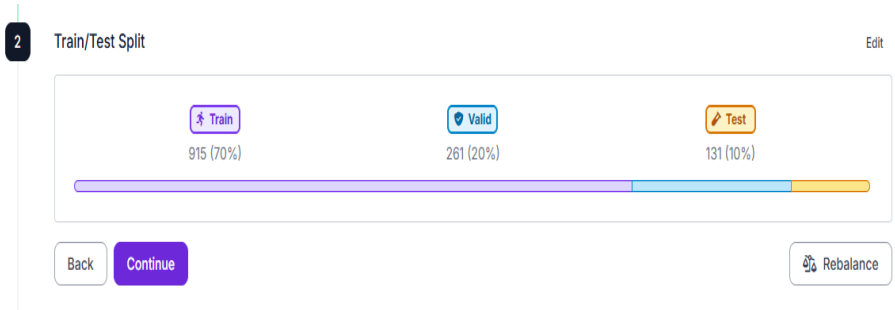
NO	Gambar Hasil *Anotate*	Abjad
3		C
4		D
5		E

Pada Tabel 4.1 menampilkan sampel hasil anotasi abjad SIBI dari huruf A sampai E, sebagai representasi dari keseluruhan proses anotasi yang dilakukan terhadap seluruh kelas huruf dalam dataset. Setiap citra pada tabel tersebut menunjukkan gestur tangan yang merepresentasikan huruf tertentu, di mana garis berwarna yang mengelilingi objek pada gambar merepresentasikan kotak anotasi yang telah dibuat untuk menandai lokasi dan batas objek tangan yang menjadi target

deteksi. Proses anotasi ini dilakukan secara konsisten pada setiap citra untuk memastikan bahwa model dapat mempelajari posisi dan karakteristik gestur tangan dengan akurat pada tahap pelatihan selanjutnya.

4.3.2 Hasil *Data Splitting*

Proses pembagian data dilakukan dengan melakukan *rebalance* dataset ke dalam tiga subset, yaitu data latih (*train*), data validasi (*validation*), dan data uji (*test*), menggunakan fitur *rebalance* yang tersedia pada platform Roboflow, dengan perbandingan sebesar 70%:20%:10%. Rasio ini dipilih karena merupakan proporsi yang umum digunakan dalam penelitian *machine learning*, di mana porsi data latih yang lebih besar (70%) diperlukan agar model memperoleh variasi data yang cukup untuk mempelajari pola dari setiap kelas huruf secara optimal, sementara data validasi (20%) digunakan untuk memantau performa model dan mendeteksi indikasi *overfitting* selama proses pelatihan, dan data uji (10%) digunakan untuk mengevaluasi performa akhir model terhadap data yang belum pernah dilihat sebelumnya. Pembagian tersebut dilakukan agar proses evaluasi model dapat berlangsung secara objektif, sekaligus memungkinkan pemantauan terhadap kemungkinan terjadinya *overfitting* selama tahap pelatihan berlangsung.

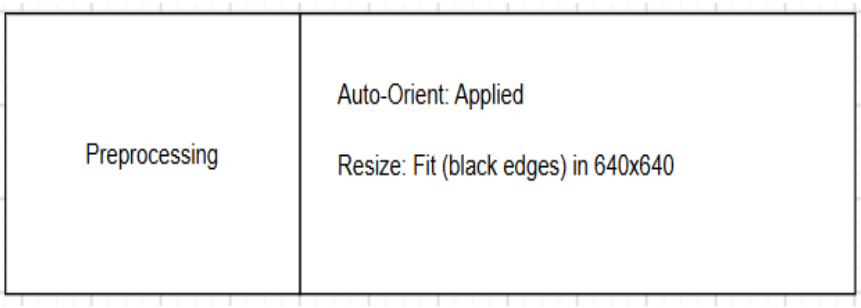


Gambar 4. 3 Hasil *Data Splitting*

Pada Gambar 4.3 menampilkan hasil pembagian dataset yang telah dilakukan sebelumnya, dengan rincian jumlah data pada masing-masing subset, yaitu sebanyak 915 citra digunakan sebagai data latih (*train*), 261 citra digunakan sebagai data validasi (*validation*), dan 131 citra digunakan sebagai data uji (*test*).

4.3.3 Hasil *Preprocessing*

Tahap berikutnya dalam penelitian ini adalah *preprocessing* atau pra-pemrosesan data. Tahapan ini bertujuan untuk mempersiapkan dan mengolah data mentah citra abjad SIBI agar layak dan siap digunakan sebagai masukan dalam proses pelatihan model deteksi objek YOLOv8. Proses *preprocessing* ini dilakukan melalui platform Roboflow, yang menyediakan berbagai fitur untuk mengelola dataset citra secara terstruktur sebelum dilanjutkan ke tahap pelatihan model.



Gambar 4. 4 Pengaturan Preprocessing

Pada gambar 4.4 menampilkan pengaturan tahap *preprocessing* pada citra, di mana diterapkan proses *auto-orient* untuk mengoreksi orientasi gambar secara otomatis, serta *resize fit (black edges)* untuk menyesuaikan ukuran citra menjadi 640x640 piksel dengan menambahkan tepi berwarna hitam pada bagian yang tidak terisi.

4.3.4 Hasil *Data Augmentasi*

Tahap berikutnya dalam penelitian ini adalah augmentasi data. Tahapan ini bertujuan untuk memperbanyak dan memperkaya variasi data citra secara artifisial tanpa harus melakukan pengumpulan data tambahan secara manual, sehingga jumlah data latih yang tersedia menjadi lebih banyak dan beragam. Proses augmentasi ini dilakukan melalui fitur *generate* pada platform Roboflow, yang menghasilkan sejumlah variasi citra baru pada subset *train* dengan tetap mempertahankan label objek pada masing-masing citra.

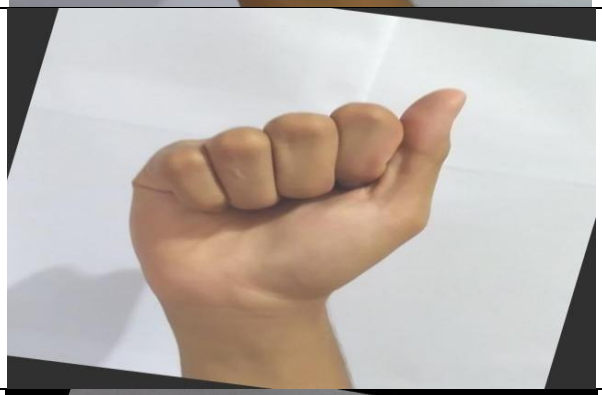
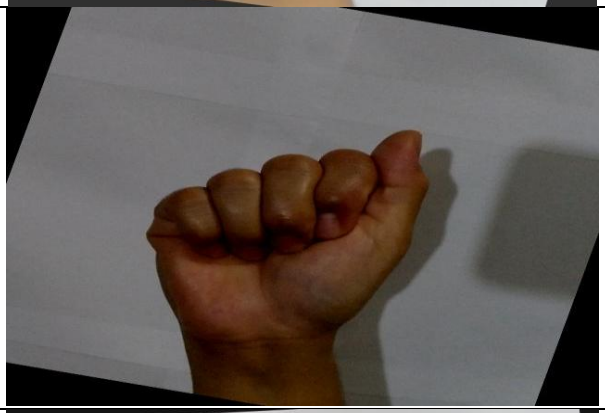
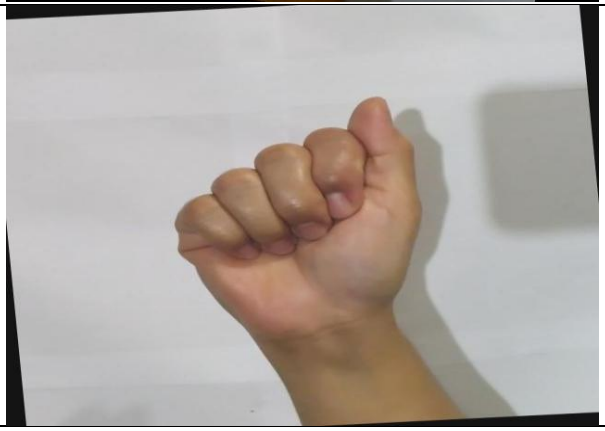
Augementation	Outputs per training example: 3 Crop: 0% Minimum Zoom, 15% Maximum Zoom Rotation: Between -15° and +15° Brightness: Between -25% and +25% Blur: Up to 2px Noise: Up to 1% of pixels

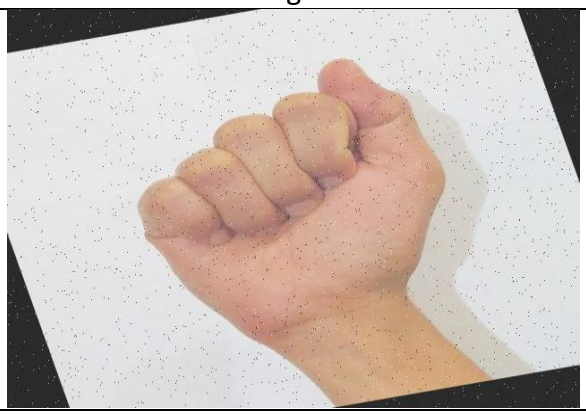
Gambar 4. 5 Pengaturan Augmentasi

Pada Gambar 4.5 memperlihatkan hasil konfigurasi tahap augmentasi data pada *Roboflow*, di mana setiap data latih menghasilkan 3 variasi keluaran. Konfigurasi augmentasi yang diterapkan meliputi *crop* dengan rentang 0% *minimum zoom* hingga 15% *maximum zoom*, rotasi citra pada rentang -15° hingga +15°, penyesuaian tingkat kecerahan antara -25% hingga +25%, penambahan efek *blur* hingga 2px, serta penambahan *noise* hingga 1% dari total piksel pada citra. Pemilihan rentang nilai pada setiap parameter augmentasi ini disesuaikan agar variasi yang dihasilkan tetap merepresentasikan kondisi pengambilan gambar yang realistis, seperti perbedaan sudut kamera, pencahayaan ruangan, maupun gangguan kecil pada kualitas citra, tanpa mengubah bentuk gestur tangan secara berlebihan sehingga label kelas abjad SIBI tetap dapat dikenali dengan baik.

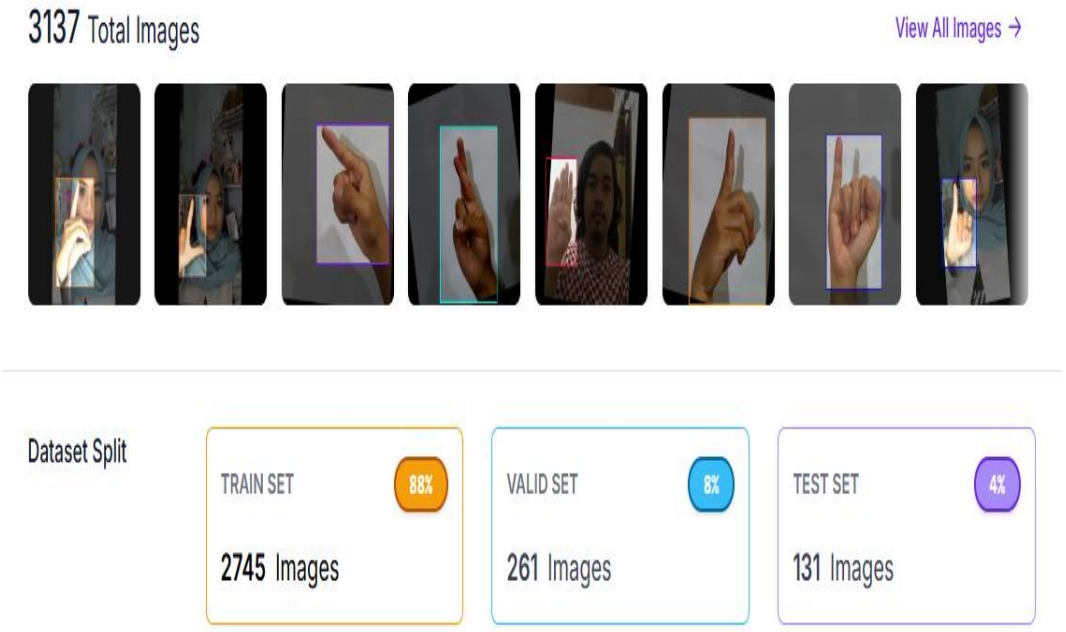
Dengan diterapkannya teknik augmentasi ini, jumlah data latih yang tersedia menjadi lebih banyak dan bervariasi dibandingkan dengan data asli sebelum augmentasi, sehingga diharapkan dapat membantu model dalam mengenali pola gestur tangan secara lebih general dan tidak hanya bergantung pada kondisi pengambilan data yang seragam. Hal ini juga bertujuan untuk mengurangi risiko *overfitting* pada saat proses pelatihan model, mengingat model akan terpapar pada variasi data yang lebih luas selama proses pembelajaran berlangsung. Setiap sampel hasil augmentasi yang diperoleh dari konfigurasi tersebut dapat dilihat pada Tabel 4.2.

Tabel 4. 2 Sample Hasil Augmentasi

NO	Hasil Augmentasi	Jenis Augmentasi
1		*Crop*
2		*Rotation*
3		*Brightness*
4		*Blur*

NO	Hasil Augmentasi	Jenis Augmentasi
5		*Noise*

Setelah proses augmentasi diterapkan, jumlah data pada subset *train* mengalami peningkatan yang cukup signifikan. Dataset yang semula hanya berjumlah 1307 citra bertambah menjadi 3137 citra, atau meningkat lebih dari dua kali lipat dari jumlah data awal. Peningkatan jumlah data ini merupakan hasil dari penerapan berbagai teknik augmentasi seperti *crop*, rotasi, penyesuaian *brightness*, *blur*, dan *noise* yang telah dijelaskan sebelumnya, di mana setiap citra asli menghasilkan beberapa variasi keluaran baru.

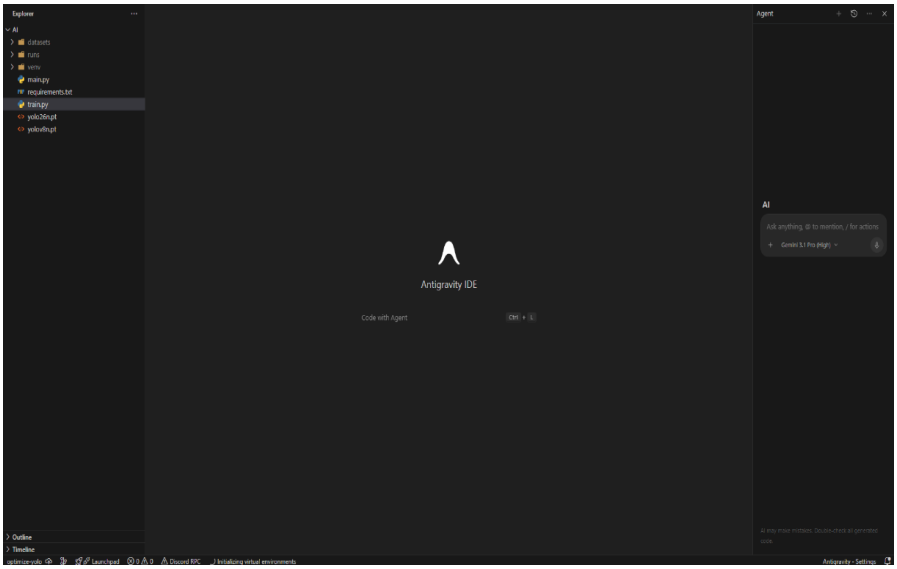


Gambar 4. 6 Jumlah Dataset setelah proses Augmentasi

4.4 Hasil *Modeling*

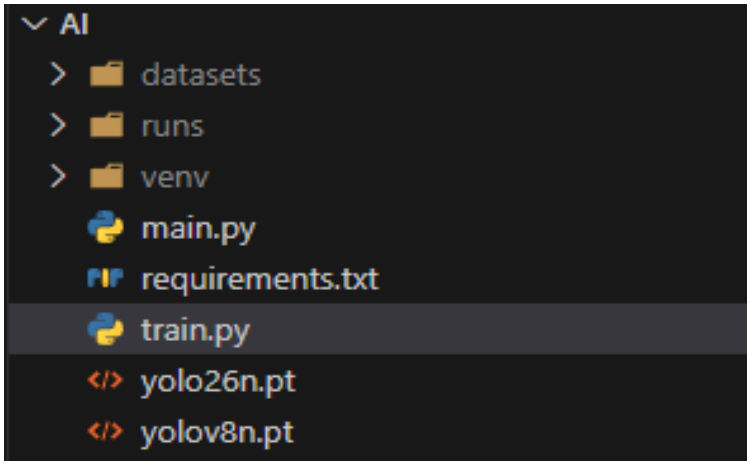
Tahap pemodelan (*modeling*) dalam penelitian ini dilaksanakan dengan tujuan utama untuk membangun sekaligus mengevaluasi kinerja dari model deteksi abjad Sistem Isyarat Bahasa Indonesia (SIBI). Pendekatan komputasi yang diterapkan pada tahap ini memanfaatkan algoritma YOLOv8 yang berfungsi sebagai metode utama dalam mengenali objek isyarat secara cepat dan presisi. Sebelum memasuki tahap pelatihan model secara mendalam, dataset citra yang berisi ragam gestur tangan telah diproses secara komprehensif melalui tahapan persiapan data (*data preparation*). Proses persiapan ini merupakan langkah awal yang sangat esensial untuk mengkondisikan kualitas dataset, sehingga model komputer yang dibangun nantinya mampu mempelajari karakteristik visual serta mengenali pola dari setiap klasifikasi abjad isyarat SIBI secara optimal dan akurat.

Sebagai langkah pendahuluan yang krusial sebelum melaksanakan proses pemodelan, persiapan lingkungan pelatihan komputasi dilakukan terlebih dahulu guna menunjang kelancaran dan stabilitas eksekusi riset. Persiapan lingkungan ini mengimplementasikan Google Antigravity, yang merupakan sebuah perangkat lunak *Integrated Development Environment* (IDE) mutakhir berbasis *agentic coding*. Penggunaan platform ini dipilih karena memungkinkan seluruh tahapan penulisan sintaks program, uji coba eksekusi perintah, hingga pengelolaan basis kode dapat dijalankan dengan tingkat efisiensi yang jauh lebih tinggi melalui dukungan asisten *Artificial Intelligence* (AI) yang terintegrasi langsung di dalam sistemnya. Adapun representasi visual dari antarmuka halaman utama perangkat lunak Google Antigravity yang digunakan selama tahapan perancangan ini dapat dilihat pada Gambar 4.7, yang memperlihatkan tata letak ruang kerja serta fungsionalitas agen cerdas dalam membantu proses penulisan kode.



Gambar 4. 7 Tampilan Halaman Antigravity

Setelah proses persiapan data selesai, tahapan selanjutnya adalah mengintegrasikan *dataset* ke dalam lingkungan kerja *Google Antigravity* agar proses pemodelan dapat dilaksanakan. *Dataset* yang telah dikonfigurasi pada *platform Roboflow* diunduh dalam format arsip ZIP. Selanjutnya, arsip tersebut diekstrak dan seluruh citra beserta labelnya ditempatkan secara langsung ke dalam direktori datasets pada struktur proyek utama. Pendekatan pengelolaan data secara lokal ini memudahkan pemanggilan direktori saat proses pelatihan model dijalankan. Struktur penempatan direktori *dataset* beserta berkas-berkas skrip *Python* pendukung lainnya di dalam ruang kerja *Google Antigravity* dapat dilihat pada Gambar 4.8.



Gambar 4. 8 Struktur Direktori Training

Tahap berikutnya setelah penempatan *dataset* adalah pelaksanaan proses pelatihan (*training*) model. Skrip kode yang digunakan untuk menjalankan pelatihan ini ditulis dalam bahasa *Python* dengan memanfaatkan pustaka *ultralytics*. Pada tahap inisialisasi, model dasar YOLOv8 dengan arsitektur berukuran *nano* (yolov8n.pt) dimuat untuk digunakan sebagai fondasi pengenalan objek.

Selanjutnya, proses pelatihan dieksekusi dengan mengatur beberapa hiperparameter (*hyperparameters*) utama. Berkas konfigurasi data.yaml yang tersimpan di dalam direktori datasets didefinisikan sebagai acuan untuk membaca struktur kelas dan lokasi citra. Pelatihan diatur agar berjalan sebanyak 100 iterasi (*epochs*) dengan resolusi masukan citra sebesar 640 piksel. Selain itu, parameter visualisasi (plots=True) diaktifkan untuk menghasilkan grafik metrik evaluasi secara otomatis. Guna mengoptimalkan dan mempercepat waktu komputasi, proses pelatihan dialokasikan pada unit pemrosesan grafis atau *Graphics Processing Unit* (GPU) melalui konfigurasi perangkat (device=0).

train.py

```
train.py > ...
You, 11 minutes ago | 2 authors (EvandriRidhoHasmono and one other)
1  from ultralytics import YOLO
2
3  def train_model():
4      # load model
5      model = YOLO('yolov8n.pt')
6
7      # train model
8      print("Mulai Proses Training....")
9      results = model.train(
10         data="datasets/data.yaml",
11         epochs=100,
12         imgsz=640,
13         plots=True,
14         device=0
15     )
16
17 if __name__ == "__main__":
18     train_model()      You, 11 minutes ago • Uncommitted changes
```

Gambar 4. 9 Skrip Pelatihan model

Pelatihan model YOLOv8 untuk *dataset* abjad Sistem Isyarat Bahasa Indonesia (SIBI) dieksekusi melalui antarmuka baris perintah. Eksekusi perintah ini dilakukan dengan menjalankan berkas train.py di dalam lingkungan kerja virtual. Proses ini bertujuan untuk menghasilkan model yang mampu mendeteksi gestur tangan ke dalam abjad yang tepat.

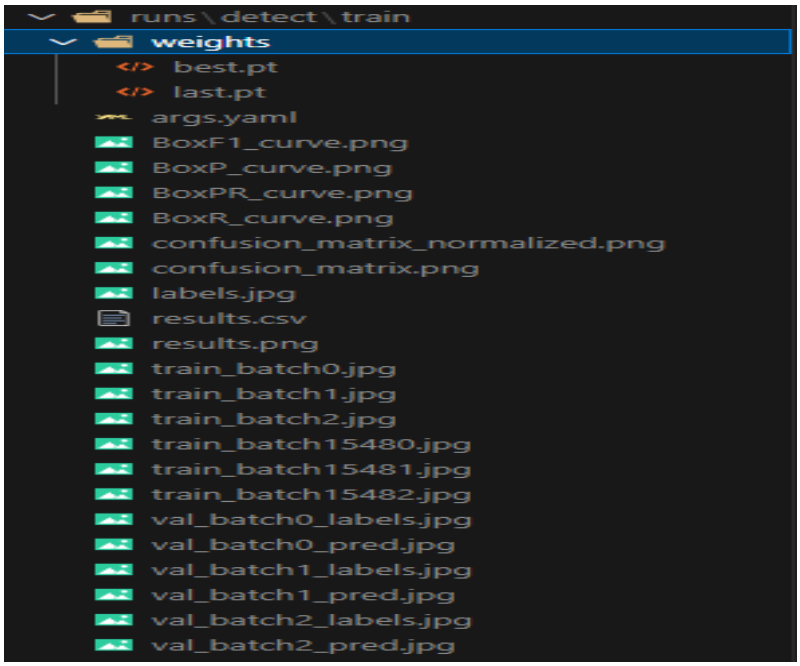
Pada tahap awal pelatihan, sistem secara otomatis melakukan inisialisasi lingkungan komputasi berdasarkan luaran yang dihasilkan pada terminal. Proses inisialisasi ini mencakup pendeteksian perangkat keras, di mana pelatihan berjalan secara optimal menggunakan akselerasi unit pemrosesan grafis, yakni NVIDIA GeForce RTX 3050 Laptop GPU. Selain itu, sistem juga melakukan penyesuaian terhadap arsitektur model bawaan. Jumlah kelas yang semula diatur sebanyak 80 kelas berdasarkan bobot awal, secara otomatis diubah menjadi 26 kelas agar sesuai dengan jumlah label abjad pada *dataset* yang dilatih. Adapun tampilan proses eksekusi pelatihan pada terminal tersebut dapat dilihat pada Gambar 4.10.

```
(venv) D:\Development\MachineLearning\SignLanguangeApp\AI>python train.py
Mulai Proses Training....
New https://pypi.org/project/ultralytics/8.4.104 available  Update with 'pip install -U ultralytics'
Ultralytics 8.4.52  Python-3.11.9 torch-2.5.1+cu121 CUDA:0 (NVIDIA GeForce RTX 3050 Laptop GPU, 4096MiB)
engine\trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugment, batch=16, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None, close_mosaic=1
0, mixup=0.0, mode=train, model=yolov8n.pt, momentum=0.937, mosaic=1.0, multi_scale=0.0, name=train, nbs=64, nms=False, opset=None, optimize=False, optimizer=auto, overlap_mask=
True, patience=100, perspective=0.0, plots=True, pose=12.0, pretrained=True, profile=False, project=None, rect=False, resume=False, retina_masks=False, rle=1.0, save=True, save_
conf=False, save_crop=False, save_dir=D:\Development\MachineLearning\SignLanguangeApp\runs\detect\train, save_frames=False, save_json=False, save_period=-1, save_txt=False, scal
e=0.5, seed=0, shear=0.0, show=False, show_boxes=True, show_conf=True, show_labels=True, simplify=True, single_cls=False, source=None, split=val, stream_buffer=False, task=detec
t, time=None, tracker=botsort.yaml, translate=0.1, val=True, verbose=True, vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0, warmup_momentum=0.8, weight_deca
y=0.0005, workers=8, workspace=None
Overriding model.yaml nc=80 with nc=26

                   from  n    params  module                                       arguments
  0                  -1  1       464  ultralytics.nn.modules.conv.Conv             [3, 16, 3, 2]
  1                  -1  1      4672  ultralytics.nn.modules.conv.Conv             [16, 32, 3, 2]
  2                  -1  1      7360  ultralytics.nn.modules.block.C2f             [32, 32, 1, True]
  3                  -1  1     18560  ultralytics.nn.modules.conv.Conv             [32, 64, 3, 2]
  4                  -1  2     49664  ultralytics.nn.modules.block.C2f             [64, 64, 2, True]
  5                  -1  1     73984  ultralytics.nn.modules.conv.Conv             [64, 128, 3, 2]
  6                  -1  2    197632  ultralytics.nn.modules.block.C2f             [128, 128, 2, True]
  7                  -1  1    295424  ultralytics.nn.modules.conv.Conv             [128, 256, 3, 2]
  8                  -1  1    460288  ultralytics.nn.modules.block.C2f             [256, 256, 1, True]
  9                  -1  1    164608  ultralytics.nn.modules.block.SPPF            [256, 256, 5]
 10                  -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 11            [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 12                  -1  1    148224  ultralytics.nn.modules.block.C2f             [384, 128, 1]
 13                  -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 14            [-1, 4]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 15                  -1  1     37248  ultralytics.nn.modules.block.C2f             [192, 64, 1]
 16                  -1  1     36992  ultralytics.nn.modules.conv.Conv             [64, 64, 3, 2]
 17           [-1, 12]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 18                  -1  1    123648  ultralytics.nn.modules.block.C2f             [192, 128, 1]
 19                  -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
 20            [-1, 9]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 21                  -1  1    493056  ultralytics.nn.modules.block.C2f             [384, 256, 1]
 22       [15, 18, 21]  1    756382  ultralytics.nn.modules.head.Detect           [26, 16, None, [64, 128, 256]]
Model summary: 130 layers, 3,015,918 parameters, 3,015,902 gradients, 8.2 GFLOPs
```

Gambar 4. 10 Proses Training Model

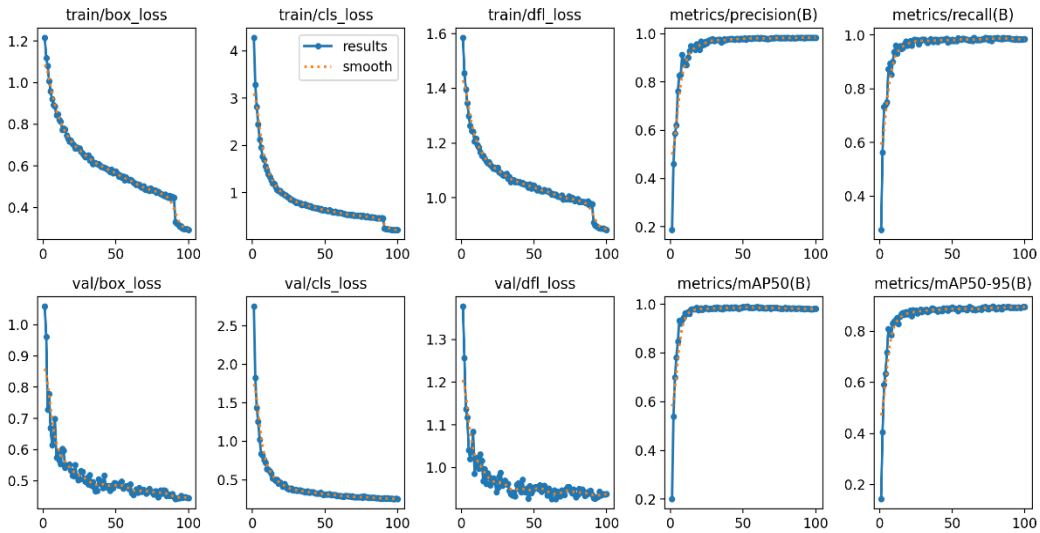
Selama proses pelatihan (*training*) berlangsung, sistem dari pustaka *Ultralytics* secara otomatis mendokumentasikan seluruh riwayat dan metrik evaluasi ke dalam sebuah direktori luaran utama, yaitu runs/detect/train. Di dalam direktori tersebut, sistem menyimpan bobot (*weights*) model yang kinerjanya dievaluasi secara berkala menggunakan data validasi. Bobot model yang menghasilkan performa akurasi terbaik selama seluruh siklus iterasi akan disimpan secara otomatis dalam format berkas best.pt, sementara bobot pada iterasi terakhir disimpan dalam berkas last.pt. Selain berkas bobot, direktori ini juga menghasilkan dan menyimpan berbagai dokumen evaluasi visual, seperti fail CSV yang berisi log metrik, kurva presisi dan *recall*, hingga *confusion matrix*, yang sangat krusial untuk analisis performa model tahap selanjutnya. Adapun struktur direktori penyimpanan hasil pelatihan beserta letak berkas bobot tersebut diilustrasikan pada Gambar 4.11.



Gambar 4. 11 Hasil Pelatihan Model

4.5 Hasil *Evaluation*

Tahap pelatihan model menghasilkan grafik yang menggambarkan *loss, precision, recall, mAP50,* dan *mAP50-95*, seperti yang ditunjukkan pada Gambar 4.12.



Gambar 4. 12 Grafik Hasil Pelatihan

Gambar 4.12 menunjukkan terjadi penurunan secara bertahap pada proses *box_loss, cls_loss, dfl_loss* baik pada proses *training* dan *validation.* Di sisi lain terdapat peningkatan pada *precision, recall, mAP50,* dan *mAP50-95,* yang menunjukkan bahwa proses pelatihan model berjalan dengan baik dan mampu mempelajari pola abjad SIBI tanpa indikasi *overfitting.*

Perbandingan hasil pelatihan model abjad SIBI dilakukan untuk melihat perkembangan performa pelatihan di beberapa *epoch*, yaitu *epoch* ke-10, 20, 30, 40, 50, 60, 70, 80, 90, dan 100, sebagaimana disajikan pada Tabel 4.3.

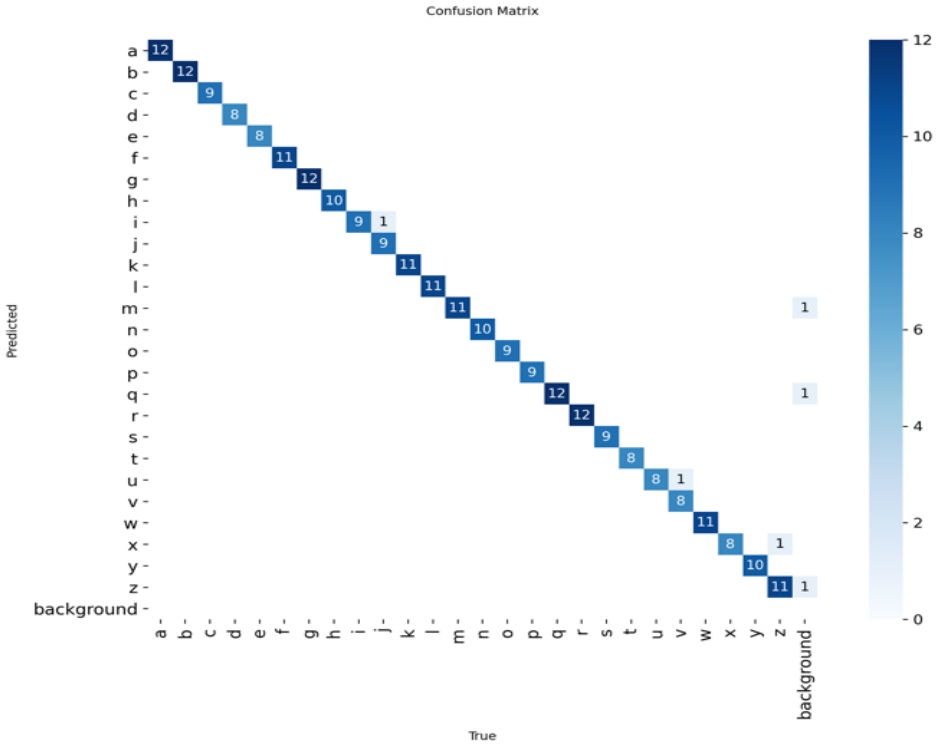
Tabel 4. 3 Perbandingan Epoch

Epoch	GPU_ mem	*box_ loss*	*cls_ loss*	*dfl_ loss*	*Box prec.*	*Box rec.*	*Box mAP50*	*Box mAP50-95*
10	1.98G	0.8493	1.483	1.217	0.871	0.937	0.962	0.837
20	1.98G	0.708	0.9958	1.12	0.934	0.971	0.977	0.87
30	1.98G	0.6475	0.7896	1.086	0.97	0.983	0.985	0.882
40	1.98G	0.5968	0.7136	1.058	0.978	0.975	0.983	0.876
50	1.98G	0.5749	0.6324	1.044	0.978	0.983	0.988	0.893
60	1.98G	0.5353	0.5822	1.026	0.984	0.983	0.988	0.883
70	1.98G	0.4879	0.5112	0.9933	0.981	0.983	0.984	0.884
80	1.98G	0.4714	0.481	0.9966	0.983	0.989	0.982	0.886
90	1.98G	0.4483	0.4557	0.9774	0.985	0.987	0.981	0.891

Epoch	GPU_ mem	*box_ loss*	*cls_ loss*	*dfl_ loss*	*Box prec.*	*Box rec.*	*Box mAP50*	*Box mAP50-95*
100	1.98G	0.2929	0.209	0.8827	0.984	0.984	0.98	0.895

Berdasarkan Tabel 4.3, dapat dilihat perbandingan hasil pelatihan model abjad SIBI pada setiap *epoch.* Pada *box_loss, cls_loss,* dan *dfl_loss* terjadi penurunan secara bertahap, sedangkan pada *precision, recall, mAP50,* dan *mAP50-95* cenderung naik meskipun terdapat beberapa titik menurun. Penurunan yang cukup signifikan pada *box_loss* terjadi setelah *epoch* ke-90, diduga karena YOLO mematikan augmentasi gambar mosaik.

Confusion matrix digunakan untuk mengevaluasi kinerja model dalam memprediksi abjad SIBI. Seperti ditunjukkan pada Gambar 4.13. *Confusion matrix* dapat menunjukkan letak kesalahan model dalam memprediksi antar kelas abjad SIBI. Visualisasi ini membantu mengamati kemampuan model secara lebih rinci.



Gambar 4. 13 Confusion Matrix Abjad SIBI

Gambar 4.13 memperlihatkan *confusion matrix* deteksi abjad SIBI, dari hasil pengamatan dapat dilihat terjadi enam kesalahan prediksi yang dilakukan oleh model. Model keliru menilai huruf ‘j’ sebagai ‘i’, huruf ‘v’ sebagai ‘u’, dan huruf ‘z’ menjadi ‘x’. Selain itu model juga salah dalam menebak latar belakang menjadi ‘m’, ‘q’, dan ‘z’

Selanjutnya adalah menentukan model dengan performa paling optimal. Pemilihan ini mengacu pada metrik evaluasi YOLOv8 untuk mengenali huruf-huruf SIBI (A-Z). Untuk melihat ringkasan kinerja model secara keseluruhan, dapat dilihat pada Gambar 4.14.

```
100 epochs completed in 1.434 hours.
Optimizer stripped from D:\Development\MachineLearning\SignLanguangeApp\runs\detect\train\weights\last.pt, 6.3MB
Optimizer stripped from D:\Development\MachineLearning\SignLanguangeApp\runs\detect\train\weights\best.pt, 6.3MB

Validating D:\Development\MachineLearning\SignLanguangeApp\runs\detect\train\weights\best.pt...
Ultralytics 8.4.52  Python-3.11.9 torch-2.5.1+cu121 CUDA:0 (NVIDIA GeForce RTX 3050 Laptop GPU, 4096MiB)
Model summary (fused): 73 layers, 3,010,718 parameters, 0 gradients, 8.1 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 9/9 3.2it/s 2.8s
                   all        261        261      0.983      0.986      0.982      0.896
                     a         12         12          1      0.992      0.995      0.883
                     b         12         12      0.993          1      0.995      0.921
                     c          9          9      0.989          1      0.995      0.878
                     d          8          8      0.988          1      0.995      0.949
                     e          8          8      0.986          1      0.995      0.871
                     f         11         11      0.991          1      0.995       0.92
                     g         12         12      0.993          1      0.995      0.867
                     h         10         10      0.992          1      0.995       0.93
                     i          9          9      0.893          1      0.895      0.885
                     j         10         10      0.993        0.9      0.948      0.802
                     k         11         11      0.991          1      0.995      0.936
                     l         11         11       0.99          1      0.995      0.978
                     m         11         11      0.988          1      0.995      0.906
                     n         10         10          1      0.927      0.995      0.905
                     o          9          9      0.996          1      0.995       0.89
                     p          9          9      0.993          1      0.995      0.856
                     q         12         12      0.985          1      0.995      0.927
                     r         12         12      0.992          1      0.995      0.905
                     s          9          9      0.988          1      0.995      0.919
                     t          8          8       0.99          1      0.995      0.897
                     u          8          8      0.882      0.935      0.898      0.863
                     v          9          9      0.988      0.889      0.908      0.848
                     w         11         11      0.994          1      0.995      0.952
                     x          8          8      0.984          1      0.995      0.852
                     y         10         10      0.992          1      0.995      0.885
                     z         12         12      0.998          1      0.995      0.874
Speed: 0.3ms preprocess, 4.7ms inference, 0.0ms loss, 1.6ms postprocess per image
Results saved to D:\Development\MachineLearning\SignLanguangeApp\runs\detect\train
```

Gambar 4. 14 Hasil model abjad SIBI

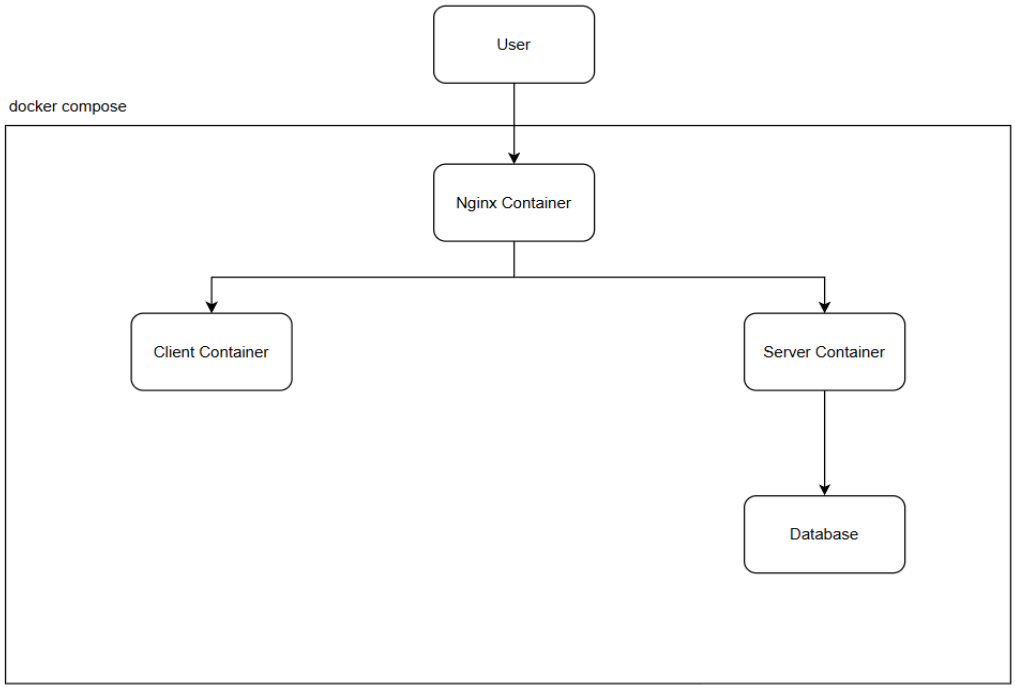
Gambar 4.14 menunjukkan hasil evaluasi model antar kelas abjad SIBI. YOLO melakukan 100 *epoch* dan menghasilkan *best.pt* dan *last.pt* yang berlangsung selama 1,434 jam. *Best.pt* diambil saat model mencapai performa atau

metrik evaluasi tertinggi, sedangkan *last.pt* pada akhir pelatihan atau *epoch* terakhir. Berdasarkan hasil Gambar 4.14 dapat dilihat terdapat *Images, Instances, Precision, Recall, mAP50, mAP50-95, preprocess, inference*, dan *postprocess*. Pada hasil evaluasi *mAP50-95* menunjukkan huruf 'l' mendapatkan *score* terbaik dengan 0,978, sedangkan huruf 'j' mendapatkan *score* terendah dengan 0,802. Selanjutnya dapat dilihat model membutuhkan 0,3ms untuk menyiapkan gambar (*preprocess)*, 4,7ms untuk memprediksi abjad (*inference*), dan 1,6ms untuk memproses hasil mentah menjadi *output (postprocess)*.

4.6 Hasil *Deployment*

4.6.1 *Deployment*

Arsitektur *system* yang dibagun dalam penelitian ini menerapkan pendekatan kontainerisasi menggunakan *docker compose* untuk mengelola beberapa layanan secara terpisah namun saling terintegrasi seperti yang ditunjukkan pada gambar 4.15.

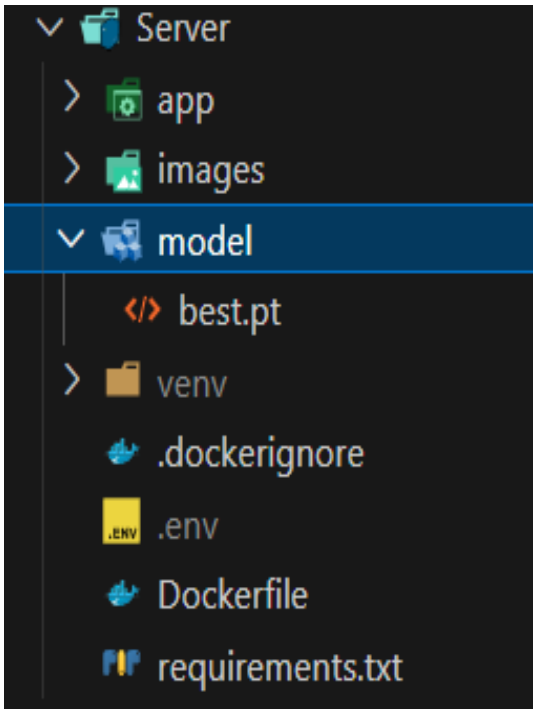


Gambar 4. 15 Arsitektur sistem

Permintaan dari pengguna pertama kali diterima oleh Nginx Container yang berfungsi sebagai *reverse proxy*, bertugas mengatur dan mendistribusikan trafik ke

dua kontainer di bawahnya, yaitu Client Container dan Server Container. Client Container menangani lapisan antarmuka (frontend) yang berinteraksi langsung dengan pengguna, sementara Server Container menangani logika bisnis dan pemrosesan data pada sisi backend. Server Container selanjutnya terhubung dengan Database untuk melakukan operasi penyimpanan, pembacaan, serta pengelolaan data yang dibutuhkan oleh sistem. Keseluruhan komponen tersebut berjalan dalam satu lingkungan Docker Compose yang sama, sehingga memudahkan proses deployment, isolasi antar layanan, serta skalabilitas sistem secara keseluruhan.

FastAPI adalah kerangka kerja berbasis bahasa pemrograman Python yang digunakan untuk membuat API dengan sangat cepat, mudah, dan modern. Integrasi dilakukan dengan memasukkan hasil pelatihan model terbaik (*best.pt)* ke direktori dari *framework* FastAPI. Direktori terdiri dari *app* tempat utama sistem bekerja, *images* tempat menyimpan gambar, *model* tempat *best.pt* berada, dan *venv* tempat menyimpan *environment*. Gambar 4.16 menunjukkan struktur direktori FastAPI.



Gambar 4. 16 Struktur Folder

Selanjutnya, dibuat kode *Python* untuk melakukan *load* model YOLO agar proses tersebut hanya dilakukan satu kali pada saat server pertama kali dijalankan. Kode implementasi *load* model ditunjukkan pada Gambar 4.17.

```python
import logging
from typing import Optional
from ultralytics import YOLO
from app.core.config import MODEL_PATH

logger = logging.getLogger(__name__)

You, 2 months ago | 1 author (You)
class ModelManager:
    _model: Optional[YOLO] = None

    @classmethod
    def load(cls, path:str = MODEL_PATH) -> None:
        if cls._model is None:
            logger.info(f"Loading YOLO model from {path}...")
            cls._model = YOLO(path)
            logger.info("Model loaded successfully.")

    @classmethod
    def get(cls) -> YOLO:
        if cls._model is None:
            raise RuntimeError("Model has not been loaded yet")
        return cls._model       You, 2 months ago • ref: implement SoC i
```

Gambar 4. 17 Implementasi Load Model

Selanjutnya, setelah model YOLO berhasil di *load* akan digunakan untuk proses *service* aplikasi. Model YOLO akan digunakan pada fungsi run_inference, fungsi ini akan memanggil model yang telah di *load* sebelumnya, dan menjalankan *pipeline* YOLO yaitu *preprocessing, forward pass,* dan *postprocessing.* Selain itu, akan mengambil hasil deteksi, kemudian akan diiterasi setiap box untuk mengestrak *label, confidence,* dan *bbox*. Kode fungsi run_inference ditunjukkan pada Gambar 4.18.

```python
def run_inference(image: Image.Image) -> list[DetectionItem]:
    model = ModelManager.get()
    results = model(image, conf=0.70, verbose=False)

    detections = []
    for result in results:
        for box in result.boxes:
            detections.append(DetectionItem(
                label=model.names[int(box.cls)],
                confidence=round(float(box.conf), 4),
                bbox=[round(v,2) for v in box.xyxy[0].tolist()]
            ))
    return detections
```

Gambar 4. 18 Implementasi Run Inference

Proses berikutnya dalam tahapan *deployment* adalah *docker image*, Yaitu cetak biru bersifat *read-only* yang berisi kode program, pustaka, dan pengaturan untuk menjalankan aplikasi. Penggunaan *docker image* diperlukan untuk mempermudah proses *deployment* karena tidak perlu melakukan banyak instalasi secara manual. Kode *docker image* ditunjukkan pada Gambar 4.19.

```
You, 5 days ago | 1 author (You)
FROM python:3.11-slim
WORKDIR /app

RUN apt-get update && apt-get install -y \
    libgl1 libglib2.0-0 \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ ./app/
COPY model/ ./model/
RUN mkdir -p images/sibi

EXPOSE 8000

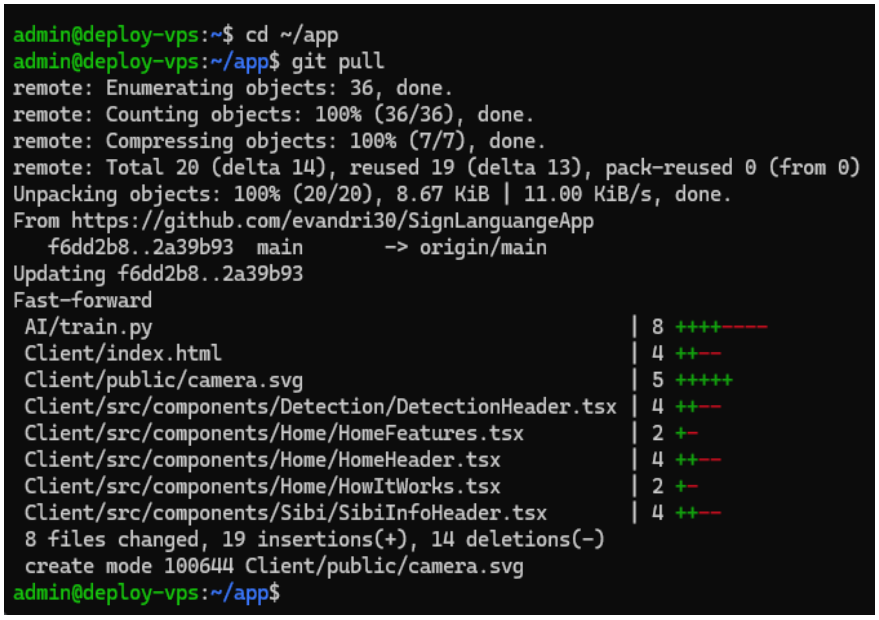
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Gambar 4. 19 Code docker image

Setelah proses *docker image* selesai, dilanjutkan dengan membuat *docker-compose*, Yaitu alat resmi untuk mendefinisikan dan menjalankan aplikasi multi-kontainer menggunakan satu file konfigurasi berformat YAML. Selanjutnya, dilakukan *commit* pada terminal menggunakan *git* agar kode tersimpan dengan aman dan riwayat pengerjaan dapat dilihat.

Tahap selanjutnya digunakan *docker* untuk proses *deployment* di vps (*virtual private server).* Untuk melakukan *deployment* dilakukan login menuju terminal vps menggunakan ssh key. Selanjutnya, dilakukan instalasi *docker* untuk menjalankan container aplikasi di server tersebut. Selanjutnya, disalin file

menggunakan *git pull* agar file dapat masuk ke dalam server. Proses *git pull* diperlihatkan pada Gambar 4.20.



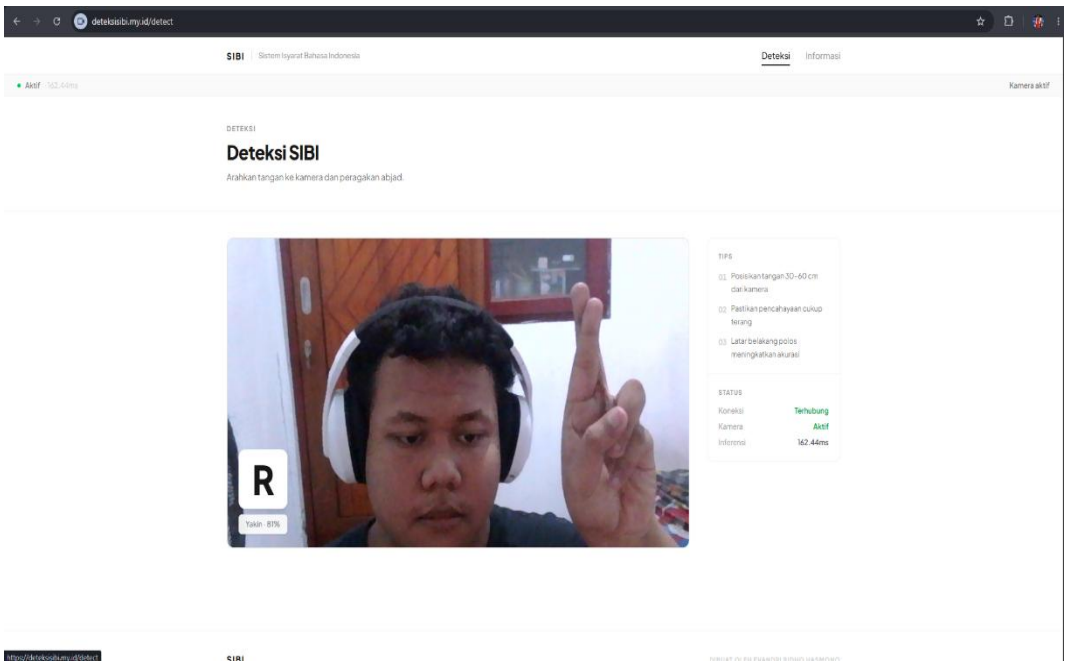
Gambar 4. 20 proses git pull

Selanjutnya dilakukan *build* di server. digunakan *docker-compose* agar dapat ditampilkan di internet. Proses *build* diperlihatkan pada Gambar 4.21.



Gambar 4. 21 Proses Build

Setelah dilakukan proses *build,* sekarang website sudah bisa diakses menggunakan internet. Penggunaan website diperlihatkan pada Gambar 4.22.



Gambar 4. 22 Tampilan website

Gambar 4.22 memperlihatkan *website* sudah bisa diakses dan digunakan secara online. Selain itu, terdapat fitur informasi untuk membantu mempelajari abjad pada SIBI. Secara keseluruhan, *website* berjalan dengan baik, meskipun masih terjadi *delay* yang disebabkan oleh *latency*, spesifikasi hardware, dan faktor lainnya.

4.6.2 Pengujian

Selain pengujian menggunakan dataset yang telah disiapkan sebelumnya, dilakukan pula pengujian tambahan menggunakan data di luar dataset setelah sistem selesai di-deploy, guna mengukur kinerja model dalam menangani data aktual yang belum pernah dilihat sebelumnya. Pengujian ini bertujuan untuk memastikan bahwa model tidak hanya bekerja baik pada data uji yang terkontrol, tetapi juga mampu memberikan hasil yang konsisten dan akurat ketika dihadapkan pada kondisi nyata di lapangan. Beberapa sampel hasil pengujian menggunakan data real tersebut ditampilkan pada Tabel 4.4.

Tabel 4. 4 Tabel pengujian

Huruf	Hasil Deteksi
A	
E	
J	
R	

Huruf	Hasil Deteksi
U	
Y	
L	

5. PENUTUP

5.1 Kesimpulan

Model YOLOv8 yang digunakan untuk mendeteksi isyarat abjad SIBI menunjukkan tingkat akurasi dan peforma yang tinggi. Model terbaik diperoleh pada epoch ke-78 dengan *Precesion* 0,9834, *Recall* 0,9862, *mAP50* 0,9823, dan *Map50-95* 0,8968. Dari sisi kecepatan, model membutuhkan waktu 0,3 ms untuk *preprocessing*, 4,7 ms untuk *interface*, dan 1,6 ms untuk *postprocess* per citra, sehingga model ini layak digunakan untuk deteksi secara *realti-time*.

Model YOLOv8 berhasil diimplementasikan untuk mendeteksi abjad SIBI dari citra tangan dengan cara diintegrasikan ke dalam sistem berbasis FastAPI sebagai backend dan ReactJs sebagai frontend. Sistem yang dibangun mampu menerima input berupa citra gestur tangan dan mengembalikan prediksi huruf SIBI secara langsung.

Metodologi CRISP-DM berhasil diterapkan dalam seluruh tahapan pengembangan model, mulai dari *business understanding* untuk mengenali tujuan, *data understanding* untuk mengenali karakteristik *dataset*, *data preparation* untuk menyiapkan data sebelum pelatihan, *modeling* untuk melakukan pelatihan model, *evaluation* untuk mengevaluasi hasil pelatihan, hingga *deployment* untuk menerapkan model ke dalam sistem berbasis web. Tahapan ini terbukti efektif dalam mengarahkan proses pengembangan model deteksi huruf SIBI secara sistematis dan terstruktur.

5.2 Saran

Berdasarkan Kesimpulan, terdapat beberapa saran:

- Menambahkan jenis gestur lain di luar abjad statis, seperti gestur angka, kata, atau kalimat, guna memperluas cakupan dan fungsionalitas sistem deteksi SIBI.
- Menggunakan versi YOLO yang lebih baru, seperti YOLOv9, untuk meningkatkan akurasi dan efisiensi performa model deteksi.

.

DAFTAR PUSTAKA

Alzubaidi, L., Zhang, J., Humaidi, A. J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M. A., Al-Amidie, M., & Farhan, L. (2021). Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data*, *8*(1). https://doi.org/10.1186/s40537-021-00444-8

Carolina Wibowo, A., & Ardi Lestari, S. (2024). *ANALISIS PENGGUNAAN MACHINE LEARNING DALAM KLASIFIKASI PENENTUAN PENYAKIT JANTUNG*. *9*(2).

Chai, J., Zeng, H., Li, A., & Ngai, E. W. T. (2021). Deep learning in computer vision: A critical review of emerging techniques and application scenarios. *Machine Learning with Applications*, *6*, 100134. https://doi.org/10.24433/CO.0411648.v1

Collins, C., Dennehy, D., Conboy, K., & Mikalef, P. (2021). Artificial intelligence in information systems research: A systematic literature review and research agenda. *International Journal of Information Management*, *60*. https://doi.org/10.1016/j.ijinfomgt.2021.102383

Darmatasia. (2021). *PENGENALAN SISTEM ISYARAT BAHASA INDONESIA (SIBI) MENGGUNAKAN GRADIENT-CONVOLUTIONAL NEURAL NETWORK DARMATASIA Teknik Informatika Fakultas Sains dan Teknologi*. *6*(1).

Dwi Grevika Drantantiyas, N., Yulita, W., Taufiq Ridwan, N., Arta Ramadhani, U., Indra Kesuma, R., Zahri Rakhman, A., Bagaskara, R., Miranto, A., & Mufidah, Z. (2023). *Performasi Deteksi Jumlah Manusia Menggunakan YOLOv8*. https://universe.roboflow.com/csgo-head-detection/head-datasets

Dwi Nurhayati, O., Eridani, D., & Hafiz Tsalavin, M. (2021). *SISTEM ISYARAT BAHASA INDONESIA (SIBI) METODE CONVOLUTIONAL NEURAL NETWORK SEQUENTIAL SECARA REAL TIME A REAL-TIME INDONESIAN LANGUAGE SIGN SYSTEM USING THE CONVOLUTION NEURAL NETWORK METHOD*. https://doi.org/10.25126/jtiik.202294787

Fatahna, I., Kusuma Sari, P. D., Kamilah, A. N., Wulanningrum, R., & Utomo, W. C. (2025). *Implementasi Computer Vision Terhadap Jenis Kualitas Pisang Susu Menggunakan Metode YOLOv8n Berbasis WebApps* (Vol. 4). https://jurnal.polsri.ac.id/index.php/jupiter/article/view/8633/3151

Gea, C., Lase, K. J. D., & Syamsudin, M. (2023). Implementasi Virtual Private Server untuk Mini Hosting. *JURNAL SAINS DAN KOMPUTER*, *7*(01), 5–9. https://doi.org/10.61179/jurnalinfact.v7i01.402

Hakim, L., Sobri, A., Sunardi, L., & Nurdiansyah, D. (2025). Prediksi penyakit jantung berbasis mesin learning dengan menggunakan metode k-nn. *Jurnal Digital Teknologi Informasi*, *7*(2), 14. https://doi.org/10.32502/digital.v7i2.9429

Investopedia. (2026). Artificial intelligence (AI): What it is and how it is used. Retrieved July 30, 2026, from https://www.investopedia.com/terms/a/artificial-intelligence-ai.asp

Isma, A. H. N., Satria, F., & Kesuma, M. E.-K. (2025). Rancang Bangun Sistem Pakar Deteksi Penyakit Daun Jeruk Menggunakan Arsitektur YOLOv9. *EXPERT: Jurnal Manajemen Sistem Informasi Dan Teknologi*, *15*(1), 65. https://doi.org/10.36448/expert.v15i1.4273

Ja'far, J., & Udjulawa, D. (2025). Deteksi Kategori Sampah Menggunakan Metode You Only Look Once. *Jurnal Algoritme*, *5*(2), 111–120. https://doi.org/10.35957/algoritme.v5i2.9047

Janiesch, C., Zschech, P., & Heinrich, K. (2021). *Machine learning and deep learning*. https://doi.org/10.1007/s12525-021-00475-2/Published

Jia, W., & Li, C. (2024). SLR-YOLO: An improved YOLOv8 network for real-time sign language recognition. *Journal of Intelligent and Fuzzy Systems*, *46*(1), 1663–1680. https://doi.org/10.3233/JIFS-235132

Jonathan, J., & Hermanto, D. (2024). Penentuan Epochs Hasil Model Terbaik: Studi Kasus Algoritma YOLOv8. *Digital Transformation Technology*, *4*(2), 792–798. https://doi.org/10.47709/digitech.v4i2.4640

Maulana, I., Rahaningsih, N., & Suprapti, T. (2023). ANALISIS PENGGUNAAN MODEL YOLOV8 (YOU ONLY LOOK ONCE) TERHADAP DETEKSI CITRA SENJATA BERBAHAYA. In *Jurnal Mahasiswa Teknik Informatika* (Vol. 7, Number 6).

Megantara, R. A., Alzami, F., Pramunendar, R. A., & Prabowo, D. P. (2022). PENGEMBANGAN DAN IMPLEMENTASI DOCKER UNTUK MEMAKSIMALKAN UTILITAS SERVER UNIVERSITAS PADA MASA COVID-19. *Transmisi*, *24*(2), 48–54. https://doi.org/10.14710/transmisi.24.2.48-54

Nabila, M. S., Herlawati, H., & Hidayat, A. (2025). Pendeteksian dan Klasifikasi Sampah pada Bank Sampah Berbasis Web Menggunakan YOLOv11.

Journal of Students' Research in Computer Science, *6*(1), 99–112. https://doi.org/10.31599/r5me0z35

Nandita, E., & Yahfizham. (2023). Komparasi Stabilitas dan Efektifitas Phyton dengan C++ Sebagai Algoritma Pemrograman Pemecahan Masalah pada Programmer Pemula. *Jurnal Arjuna : Publikasi Ilmu Pendidikan, Bahasa Dan Matematika*, *1*(6), 104–115. https://doi.org/10.61132/arjuna.v1i6.298

Nurfatih, M. S., Nurmalasari, Y., & Prakarsyah, A. (2024). Analysis of accreditation's impact on student numbers in South Sumatra private universities using K-means clustering. ResearchGate. https://www.researchgate.net/publication/381834981_Analysis_of_Accreditation's_Impact_on_Student_Numbers_in_South_Sumatra_Private_Universities_Using_K-Means_Clustering

Putra, R. R., Maimunah, M., & Sasongko, D. (2024). Implementasi Algoritma YOLO V8 (You Only Look Once) Dalam Deteksi Penyakit Daun Durian. *Building of Informatics, Technology and Science (BITS)*, *6*(3). https://doi.org/10.47065/bits.v6i3.6136

Rachmawati, R. (2020). BAHASA ISYARAT DAN JURU BAHASA ISYARAT. In *JURNAL PENERJEMAHAN* (Vol. 7, Number 2).

Rianti, A., Majid, N. W. A., & Fauzi, A. (2023). *CRISP-DM: Metodologi Proyek Data Science*.

Ruswanti, D., Susilo, D., & Riani, R. (2024). Implementasi CRISP-DM pada Data Mining untuk Melakukan Prediksi Pendapatan dengan Algoritma C.45. *Go Infotech: Jurnal Ilmiah STMIK AUB*, *30*(1), 111–121. https://doi.org/10.36309/goi.v30i1.266

Sarker, I. H. (2022). AI-Based Modeling: Techniques, Applications and Research Issues Towards Automation, Intelligent and Smart Systems. In *SN Computer Science* (Vol. 3, Number 2). Springer. https://doi.org/10.1007/s42979-022-01043-x

Schröer, C., Kruse, F., & Gómez, J. M. (2021). A systematic literature review on applying CRISP-DM process model. *Procedia Computer Science*, *181*, 526–534. https://doi.org/10.1016/j.procs.2021.01.199

Sudriyanto, Ali Hafid, M., & Ade Kurniawan, M. (2024). Deteksi Akun Kaggle Bot Menggunakan Linear Regression. *Journal Homepage: Journal of Electrical Engineering and Computer (JEECOM)*, *6*(2). https://doi.org/10.33650/jeecom.v4i2

Suharjito, Thiracitta, N., & Gunawan, H. (2021). Digital transformation as distributed leadership: Firing the change agent. *Procedia Computer Science*, *196*, 245–254. https://doi.org/10.1016/j.procs.2021.12.011

Tristianto, D., & Limantara, M. A. (2024). SISTEM PEMBELAJARAN ISYARAT BAHASA INDONESIA (SIBI) MENGGUNAKAN METODE CONVOLUTIONAL NEURAL NETWORK (CNN). In *Surabaya Jurnal Sistem Cerdas dan Rekayasa (JSCR)* (Vol. 6, Number 2).

Ua, A. M. T. I. S., H, D. L., Marpaung, E. S. K., Ong, J., Savinka, M., Nurhaliza, P., & Ningsih, R. Y. (2023). Penggunaan Bahasa Pemrograman Python Dalam Analisis Faktor Penyebab Kanker Paru-Paru. *Jurnal Publikasi Teknik Informatika*, *2*(2), 88–99. https://doi.org/10.55606/jupti.v2i2.1742

Yayasan Peduli Kasih ABK. (2023). Jangan sampai tertukar, ini perbedaan bahasa isyarat BISINDO dan SIBI. DAAI TV. https://daaitv.co.id/DAAI-WP/jangan-sampai-tertukar-ini-perbedaan-bahasa-isyarat-bisindo-dan-sibi/

LAMPIRAN LISTING PROGRAM

```
from ultralytics import YOLO

def train_model():
    # load model
    model = YOLO('yolov8n.pt')

    # train model
    print("Mulai Proses Training....")
    results = model.train(
        data="datasets/data.yaml",
        epochs=100,
        imgsz=640,
        plots=True,
        device=0
    )

if __name__ == "__main__":
    train_model()
```

```
import os
from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL")

engine = create_engine(DATABASE_URL, pool_pre_ping=True)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
```

```
        db.close()
```

```
import logging
from typing import Optional
from ultralytics import YOLO
from app.core.config import MODEL_PATH

logger = logging.getLogger(__name__)

class ModelManager:
    _model: Optional[YOLO] = None

    @classmethod
    def load(cls, path:str = MODEL_PATH) -> None:
        if cls._model is None:
            logger.info(f"Loading YOLO model from {path}...")
            cls._model = YOLO(path)
            logger.info("Model loaded successfully.")

    @classmethod
    def get(cls) -> YOLO:
        if cls._model is None:
            raise RuntimeError("Model has not been loaded yet")
        return cls._model
```

```
from sqlalchemy import Column, Integer, String, Text, JSON
from app.core.database import Base

class SibiLetter(Base):
    __tablename__ = "sibi_letters"

    id = Column(Integer, primary_key=True, index=True)
    letter = Column(String(1), unique=True, index=True, nullable=False)
    name = Column(String(50), nullable=False)
    description = Column(Text, nullable=False)
    gesture_steps = Column(JSON, nullable=True)
```

```
import json
import logging
from fastapi import APIRouter, WebSocket, WebSocketDisconnect
from app.services.detection import process_frame
from app.ml.manager import ModelManager

logger = logging.getLogger(__name__)
```

```
router = APIRouter()

# Connection Manager
class ConnectionManager:
    def __init__(self):
        self._active: dict[str, WebSocket] = {}

    async def connect(self, client_id: str, ws: WebSocket):
        await ws.accept()
        self._active[client_id] = ws
        logger.info(f"Client {client_id} connected. Total: {len(self._active)}")

    def disconnect(self, client_id: str):
        self._active.pop(client_id, None)
        logger.info(f"Client {client_id} disconnected.")

    @property
    def count(self):
        return len(self._active)

manager = ConnectionManager()

@router.get("/", tags=["Health"])
def health_check():
    return {"message": "API is running.", "active_connections": manager.count}

@router.get("/model-info", tags=["Health"])
def model_info():
    model = ModelManager.get()
    return {"classes": model.names, "num_classes": len(model.names)}

@router.websocket("/ws/detect/{client_id}")
async def websocket_detect(ws: WebSocket, client_id: str):
    await manager.connect(client_id, ws)
    try:
        while True:
            raw = await ws.receive_text()

            try:
                payload = json.loads(raw)
            except json.JSONDecodeError:
                await ws.send_json({"status": "error", "message": "Invalid JSON"})
                continue

            b64_frame = payload.get("frame")
            frame_id = payload.get("frame_id")
```

```
            if not b64_frame:
                await ws.send_json({"status": "error", "message": "Missing 'frame'"})
                continue

            try:
                response = await process_frame(b64_frame, frame_id)
                await ws.send_json(response.model_dump())
            except WebSocketDisconnect:
                raise
            except Exception as exc:
                logger.warning(f"Error for client {client_id}: {exc}")
                try:
                    await ws.send_json({"status": "error", "message": str(exc)})
                except Exception:
                    pass

    except WebSocketDisconnect:
        pass
    finally:
        manager.disconnect(client_id)
```

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from app.core.database import get_db
from app.models.sibi import SibiLetter
from app.schemas.sibi import SibiLetterResponse
from app.services.sibi_seeder import seed_sibi_data
from typing import List

router = APIRouter(prefix="/sibi", tags=["SIBI Info"])

@router.get("/", response_model=List[SibiLetterResponse])
def get_all_sibi_letters(db: Session = Depends(get_db)):
    """Mendapatkan daftar semua huruf SIBI A-Z"""
    letters = db.query(SibiLetter).order_by(SibiLetter.letter.asc()).all()
    return letters

@router.get("/{letter}", response_model=SibiLetterResponse)
def get_sibi_letter_detail(letter: str, db: Session = Depends(get_db)):
    """Mendapatkan informasi detail dan langkah-langkah isyarat untuk huruf tertentu."""
    if len(letter) != 1:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
```

```
            detail="Parameter letter harus berupa satu karakter tunggal."
        )

    char = letter.lower()
    db_letter = db.query(SibiLetter).filter(SibiLetter.letter == char).first()
    if not db_letter:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Informasi SIBI untuk huruf '{letter}' tidak ditemukan."
        )
    return db_letter

@router.post("/seed", status_code=status.HTTP_201_CREATED)
def trigger_seeding(db: Session = Depends(get_db)):
    """Seeding data SIBI A-Z"""
    seed_sibi_data(db)
    return {"message": "Seeding data SIBI berhasil"}
```

```
from pydantic import BaseModel
from typing import Optional

class DetectionItem(BaseModel):
    label: str
    confidence: float
    bbox: list[float]

class DetectionResponse(BaseModel):
    status: str
    frame_id: Optional[int]
    inference_ms: float
    detections: list[DetectionItem]
    top_prediction: Optional[DetectionItem]
```

```
from pydantic import BaseModel
from typing import List, Optional

class SibiLetterResponse(BaseModel):
    id: int
    letter: str
    name: str
    description: str
    gesture_steps: Optional[List[str]] = None

    class Config:
```

```
        from_attributes = True
```

```
import asyncio
import base64
import io
import time
from PIL import Image
from app.ml.manager import ModelManager
from app.schemas.detection import DetectionItem, DetectionResponse

def decode_frame(b64_data: str) -> Image.Image:
    if "," in b64_data:
        b64_data = b64_data.split(",", 1)[1]
    raw = base64.b64decode(b64_data)
    return Image.open(io.BytesIO(raw)).convert("RGB")

def run_inference(image: Image.Image) -> list[DetectionItem]:
    model = ModelManager.get()
    results = model(image, conf=0.70, verbose=False)

    detections = []
    for result in results:
        for box in result.boxes:
            detections.append(DetectionItem(
                label=model.names[int(box.cls)],
                confidence=round(float(box.conf), 4),
                bbox=[round(v,2) for v in box.xyxy[0].tolist()]
            ))
    return detections

async def process_frame(b64_frame: str, frame_id: int | None) ->
DetectionResponse:
    """Memproses frame video menggunakan model deteksi objek"""
    t0 = time.perf_counter()
    image = await asyncio.to_thread(decode_frame, b64_frame)
    detections = await asyncio.to_thread(run_inference, image)
    elapsed_ms = (time.perf_counter() - t0) * 1000

    top = max(detections, key=lambda d: d.confidence) if detections else None
    return DetectionResponse(
        status="ok",
        frame_id=frame_id,
        inference_ms=round(elapsed_ms, 2),
        detections=detections,
        top_prediction=top,)
```

```
import logging
import os
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from app.ml.manager import ModelManager
from app.routers import detection, sibi
from app.core.database import Base, engine, SessionLocal
from app.services.sibi_seeder import seed_sibi_data
from dotenv import load_dotenv

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
)

@asynccontextmanager
async def lifespan(app: FastAPI):
    logging.info("Memulai inisialisasi basis data PostgreSQL...")
    try:
        Base.metadata.create_all(bind=engine)
        db = SessionLocal()
        try:
            seed_sibi_data(db)
        finally:
            db.close()
    except Exception as e:
        logging.error(f"Gagal menginisialisasi basis data: {e}")
            logging.warning("Melanjutkan startup tanpa koneksi database yang
sukses.")

    ModelManager.load()
    yield

app = FastAPI(
    title="Sign Language Detection API",
    version="2.0.0",
    lifespan=lifespan,
)

os.makedirs("images/sibi", exist_ok=True)

app.mount("/images", StaticFiles(directory="images"), name="images")
```

```
load_dotenv()

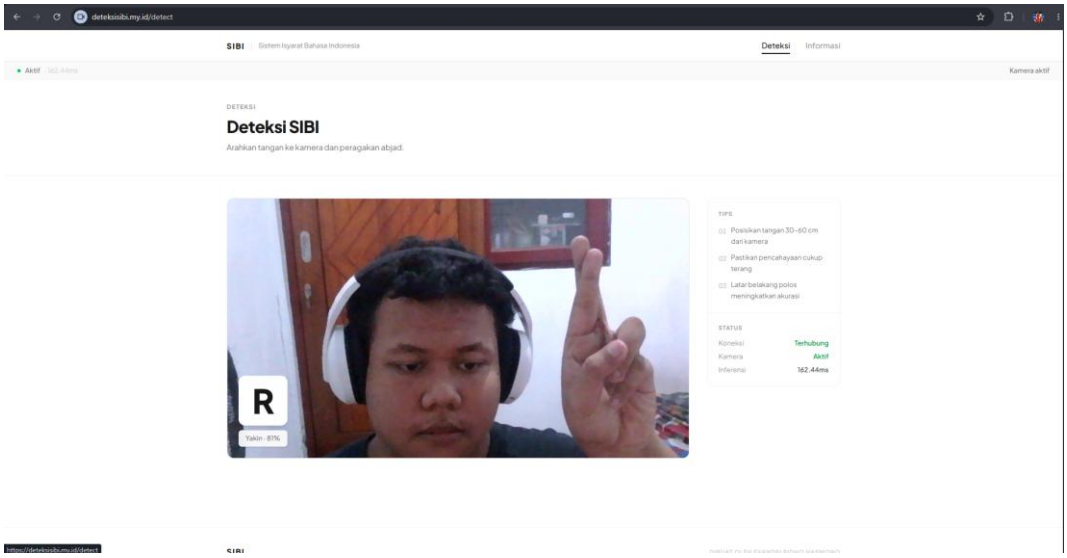
cors_origins_env = os.getenv("CORS_ORIGINS", "http://localhost:5173")
origins = [origin.strip() for origin in cors_origins_env.split(",") if origin.strip()]

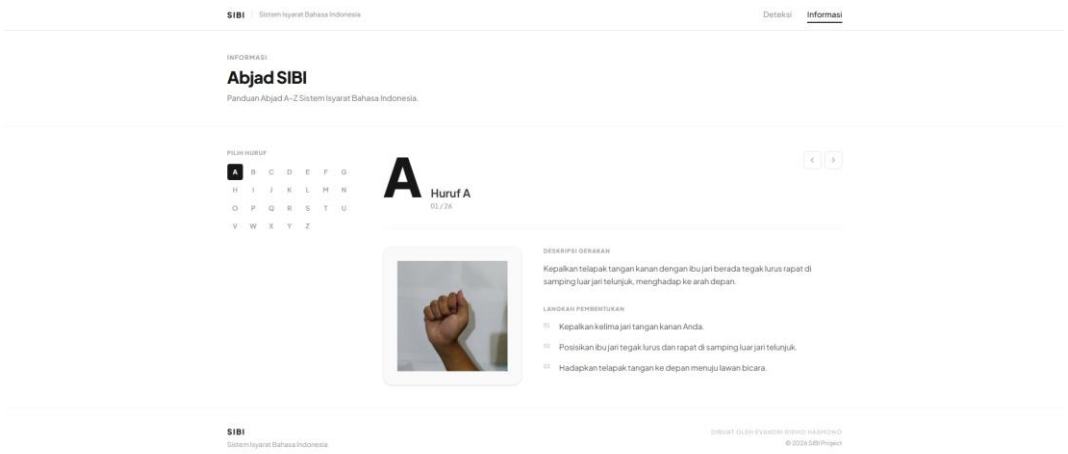
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(detection.router)
app.include_router(sibi.router, prefix="/api")
```

LAMPIRAN OUTPUT PROGRAM






SIBI
Sistem Isyarat Bahasa Indonesia
Deteksi
Informasi
INFORMASI
Abjad SIBI
Panduan Abjad A–Z Sistem Isyarat Bahasa Indonesia.
PILIH HURUF
A B C D E F G
H I J K L M N
O P Q R S T U
V W X Y Z
A
Huruf A
01 / 26
DESKRIPSI GERAKAN
Kepalkan telapak tangan kanan dengan ibu jari berada tegak lurus rapat di samping luar jari telunjuk, menghadap ke arah depan.
LANGKAH PEMBENTUKAN
01 Kepalkan kelima jari tangan kanan Anda.
02 Posisikan ibu jari tegak lurus dan rapat di samping luar jari telunjuk.
03 Hadapkan telapak tangan ke depan menuju lawan bicara.
SIBI
Sistem Isyarat Bahasa Indonesia
DIBUAT OLEH EVANDRI RISMO HASMONO
© 2026 SIBI Project